Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Уральский федеральный университет имени первого Президента России Б.Н. Ельцина»
Институт радиоэлектроники и информационных технологий - РТФ
Школа бакалавриата

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА**

на тему

# «Проектирование и разработка инструмента статико-динамического анализа EH/Unwind-метаданных Windows PE32+ AMD64»

Руководитель: _____

Нормоконтролер: _____

Студент группы _____: _____

Екатеринбург
2026

**ЗАДАНИЕ**

на выполнение выпускной квалификационной работы

Тема выпускной квалификационной работы: «Проектирование и разработка инструмента статико-динамического анализа EH/Unwind-метаданных Windows PE32+ AMD64».

Исходные данные к работе: исходный код инструмента EH/Unwind Analyzer, документация проекта, результаты статической и динамической проверки, материалы методических указаний по выполнению ВКР, требования к оформлению пояснительной записки и презентации.

Содержание работы: анализ предметной области и существующих средств исследования PE-файлов; формирование требований к инструменту; проектирование архитектуры программного средства; разработка модулей парсинга, статической оценки, динамического сбора и формирования отчетов; проверка корректности работы на тестовых и реальных наборах PE-файлов.

Перечень графического материала: структурная схема программного средства, схема статико-динамического анализа, схема динамического цикла наблюдения, таблица требований, таблица модулей, таблица результатов проверки.

Дата выдачи задания: _____

Срок сдачи работы: _____

# КАЛЕНДАРНЫЙ ПЛАН

Таблица 1 - Календарный план выполнения работы

| N | Этап | Срок |
|---|---|---|
| 1 | Анализ предметной области и источников | февраль - март 2026 |
| 2 | Формирование требований и проектирование архитектуры | март 2026 |
| 3 | Разработка парсера и статического анализа | март - апрель 2026 |
| 4 | Разработка динамического сбора и отчетов | апрель - май 2026 |
| 5 | Тестирование, оформление ПЗ и презентации | май - июнь 2026 |

Руководитель: _____

Обучающийся: _____

## РЕФЕРАТ

Пояснительная записка содержит 62 с. без приложений, 3 рисунка, 23 таблицы, 30 использованных источников, 14 приложений.

EH/UNWIND-МЕТАДАННЫЕ, PE32+, AMD64, RUNTIME_FUNCTION, UNWIND_INFO, СТАТИЧЕСКИЙ АНАЛИЗ, ДИНАМИЧЕСКИЙ АНАЛИЗ, WINDOWS DEBUG API, ЭВРИСТИЧЕСКАЯ ОЦЕНКА, JSON-ОТЧЕТ, GHIDRA.

Объектом исследования являются методы анализа служебных метаданных обработки исключений и раскрутки стека в исполняемых файлах Windows PE32+ AMD64. Предметом исследования являются алгоритмы безопасного разбора таблицы исключений, декодирования структур UNWIND_INFO, выявления структурных аномалий и сопоставления статических признаков с событиями выполнения.

Цель работы - спроектировать и разработать программный инструмент, который автоматически извлекает EH/Unwind-метаданные, выявляет потенциально некорректные или подозрительные структуры и формирует проверяемые отчеты для специалиста по анализу исполняемых файлов.

В результате работы разработано программное средство, включающее парсер PE32+ AMD64, декодер RUNTIME_FUNCTION и UNWIND_INFO, блок статической эвристической оценки, динамический сборщик событий Windows и генератор отчетов в форматах JSON, HTML, CSV и Ghidra CSV. Проверка выполнена на модульных тестах, синтетических PE-файлах с известными аномалиями, наборе обычных PE-файлов и динамических сценариях Windows.

**СОДЕРЖАНИЕ**

**РЕФЕРАТ** ..... 4

**СОДЕРЖАНИЕ** ..... 5

**ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ** ..... 8

**ПЕРЕЧЕНЬ ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ** ..... 9

**ВВЕДЕНИЕ** ..... 10

1 Анализ предметной области и формирование требований ..... 13

1.1 Общая характеристика предметной области ..... 13

1.2 EH/Unwind-метаданные PE32+ AMD64 ..... 14

1.3 Статический и динамический подходы к анализу ..... 15

1.4 Анализ существующих средств ..... 17

1.5 Требования к разрабатываемому инструменту ..... 18

1.6 Критерии корректности выявления аномалий ..... 19

1.7 Роль объяснимости в инструменте анализа ..... 20

2 Проектирование и разработка инструмента ..... 22

2.1 Выбор инструментов разработки ..... 22

2.2 Общая архитектура программного средства ..... 23

2.3 Модель входных и выходных данных ..... 24

2.4 Алгоритм парсинга PE32+ AMD64 ..... 26

2.5 Декодирование RUNTIME_FUNCTION и UNWIND_INFO ..... 27

2.6 Эвристическая модель оценки аномалий ..... 28

2.7 Динамический сбор событий Windows ..... 29

2.8 Формирование отчетов и интеграция с Ghidra ..... 30

2.9 Границы доверия и безопасность ..... 32

2.10 Обработка ошибок и отказоустойчивость ..... 33

2.11 Связь модулей с требованиями ..... 34
3 Проверка работы и оценка результатов ..... 36
3.1 Структура реализации ..... 36
3.2 Пользовательский сценарий работы ..... 37
3.3 Методика тестирования ..... 38
3.4 Результаты статической проверки ..... 39
3.5 Результаты динамической проверки ..... 40
3.6 Оценка соответствия требованиям ..... 41
3.7 Ограничения разработанного инструмента ..... 42
3.8 Практическое применение и развитие ..... 43
3.9 Интерпретация результатов анализа ..... 44
3.10 Готовность проекта к демонстрации и дальнейшему сопровождению ..... 45
3.11 Проверка полноты выполнения требований ..... 48
3.12 Сценарий демонстрации проекта на защите ..... 49
3.13 Практическая значимость разработанного инструмента ..... 50
3.14 Подготовка материалов к эксплуатации ..... 51
3.15 Анализ достаточности тестового покрытия ..... 52
3.16 Оценка качества пользовательского результата ..... 53
3.17 Сравнение достигнутого результата с первоначальной целью ..... 54
3.18 Рекомендации по представлению работы на нормоконтроле ..... 55
3.19 Оценка рисков внедрения ..... 56
3.20 Итоговая оценка готовности программного средства ..... 57
**ЗАКЛЮЧЕНИЕ** ..... 58
**СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ** ..... 61

**ПРИЛОЖЕНИЕ А**..... 63

**ПРИЛОЖЕНИЕ Б**..... 64

**ПРИЛОЖЕНИЕ В**..... 65

**ПРИЛОЖЕНИЕ Г**..... 66

**ПРИЛОЖЕНИЕ Д**..... 67

**ПРИЛОЖЕНИЕ Е**..... 68

**ПРИЛОЖЕНИЕ Ж**..... 69

**ПРИЛОЖЕНИЕ И**..... 70

**ПРИЛОЖЕНИЕ К**..... 71

**ПРИЛОЖЕНИЕ Л**..... 72

**ПРИЛОЖЕНИЕ М**..... 73

**ПРИЛОЖЕНИЕ Н**..... 74

**ПРИЛОЖЕНИЕ П**..... 75

**ПРИЛОЖЕНИЕ Р**..... 76

## ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе применяются следующие термины с соответствующими определениями:

**PE -** формат исполняемых файлов Windows, содержащий заголовки, таблицы данных, секции и служебные структуры, необходимые загрузчику операционной системы.

**PE32+ -** вариант формата PE для 64-разрядных образов, используемый в том числе для архитектуры AMD64.

**RVA -** относительный виртуальный адрес, задаваемый как смещение от базового адреса загруженного образа.

**RUNTIME_FUNCTION -** запись таблицы исключений AMD64, описывающая диапазон функции и ссылку на структуру UNWIND_INFO.

**UNWIND_INFO -** структура Windows x64 ABI, описывающая восстановление стекового кадра, сохраненные регистры, обработчики исключений и цепочки записей.

**Статический анализ -** исследование файла без его запуска, основанное на чтении байтов, проверке границ, декодировании структур и поиске противоречий.

**Динамический анализ -** наблюдение за поведением исследуемой программы во время выполнения; в данной работе выполняется только на Windows x64.

**Эвристический признак -** проверяемый технический факт, который сам по себе не доказывает вредоносность, но помогает выделить объект для ручной проверки.

**Корреляция -** сопоставление статических признаков с событиями выполнения, позволяющее усилить или ослабить аналитический вывод.

## ПЕРЕЧЕНЬ ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

В настоящей выпускной квалификационной работе применяются следующие сокращения и обозначения:

**ABI -** Application Binary Interface - бинарный интерфейс приложения

**API -** Application Programming Interface - программный интерфейс

**CSV -** Comma-Separated Values - табличный текстовый формат

**EH -** Exception Handling - обработка исключений

**JSON -** JavaScript Object Notation - формат структурированных данных

**RIP -** Instruction Pointer register - регистр адреса текущей инструкции AMD64

**SEH -** Structured Exception Handling - структурная обработка исключений Windows

**UEF -** Unhandled Exception Filter - фильтр необработанных исключений

**VCH -** Vectored Continue Handler - векторный обработчик продолжения

**VEH -** Vectored Exception Handler - векторный обработчик исключений

# ВВЕДЕНИЕ

Исполняемые файлы Windows содержат не только машинный код, импортируемые функции и пользовательские данные. Существенная часть их поведения определяется служебными структурами, которые используются загрузчиком, отладчиком, механизмом обработки исключений и средствами восстановления стека. Для 64-разрядных PE-файлов архитектуры AMD64 особенно важны таблица исключений .pdata и связанные с ней структуры .xdata. Они описывают границы функций, правила восстановления стекового кадра, сохраненные регистры и возможные обработчики исключений [1].

При обычной разработке эти структуры создаются компилятором и компоновщиком автоматически. Разработчик редко обращается к ним напрямую, поскольку в нормальном сценарии они являются технической частью контракта между компилятором, загрузчиком и средой выполнения. Однако при анализе неизвестных, поврежденных, защищенных или нестандартно собранных программ эти данные становятся самостоятельным источником сведений. По ним можно восстановить карту функций, понять, как программа возвращает управление после исключения, и заметить несоответствия между ожидаемой и фактической структурой образа [2].

Актуальность темы связана с тем, что практический анализ PE-файлов часто выполняется в условиях неполной информации. Дизассемблер показывает машинный код, PE-просмотрщик показывает таблицы и секции, отладчик показывает поведение во время запуска, но отдельная проверка EH/Unwind-слоя обычно остается ручной работой специалиста. Если таблица функций повреждена, если запись UNWIND_INFO указывает за границы секции, если обработчик расположен в неожиданной области памяти, то такая ситуация должна быть выявлена явно, а не теряться среди общего вывода инструмента.

Проблемная ситуация состоит в отсутствии простого специализированного средства, которое объединяет безопасный статический разбор EH/Unwind-метаданных, объяснимую эвристическую оценку и, при

необходимости, динамическое наблюдение за механизмами исключений Windows. Существующие инструменты полезны, но чаще решают более широкие задачи: дизассемблирование, отладку, просмотр PE-структуры или поведенческий анализ. В результате низкоуровневый слой обработки исключений оказывается либо скрыт, либо представлен в форме, требующей значительной ручной интерпретации.

Объектом исследования являются методы анализа служебных структур обработки исключений и раскрутки стека в исполняемых файлах Windows PE32+ AMD64. Предметом исследования являются алгоритмы безопасного разбора .pdata и .xdata, выявления структурных аномалий, формирования объяснимых выводов и сопоставления статических признаков с динамическими событиями Windows.

Цель выпускной квалификационной работы - проектирование и разработка инструмента статико-динамического анализа EH/Unwind-метаданных Windows PE32+ AMD64, позволяющего автоматически извлекать служебные структуры, выявлять потенциальные аномалии и формировать отчеты, пригодные для последующей ручной проверки.

Для достижения цели поставлены следующие задачи: проанализировать формат PE32+ AMD64 и роль EH/Unwind-метаданных; сформировать требования к инструменту анализа; спроектировать архитектуру программного средства и модель данных отчета; реализовать парсер, статическую оценку, динамический сбор и генерацию отчетов; проверить корректность работы на модульных тестах, синтетических образцах, обычных PE-файлах и динамических сценариях.

Методами исследования являются анализ технической документации Microsoft и спецификаций формата PE, сравнительный анализ существующих средств, структурное проектирование, алгоритмическое моделирование, экспериментальная проверка на тестовых наборах, модульное тестирование, проверка JSON-схемы и сопоставление статических и динамических результатов.

Теоретическую основу работы составляют документация Microsoft по формату PE и обработке исключений x64, описание структур RUNTIME_FUNCTION и UNWIND_INFO, документация Windows Debug API, материалы по архитектуре Intel 64, стандарты оформления программной документации и практическая литература по анализу исполняемых файлов [1]-[12].

Практическая значимость работы заключается в создании инструмента, который может использоваться в учебной лаборатории, при первичном исследовании PE-файлов и при подготовке данных для дальнейшего анализа в Ghidra. Инструмент не подменяет специалиста и не объявляет файл вредоносным автоматически, а формирует воспроизводимую цепочку технических фактов, которую можно проверить вручную.

Пояснительная записка состоит из введения, трех разделов, заключения, списка использованных источников и приложений. В первом разделе рассмотрены предметная область, существующие средства и требования к разрабатываемому инструменту. Во втором разделе описаны проектные решения, архитектура и алгоритмы. В третьем разделе приведены сведения о реализации, методике проверки и результатах экспериментальной оценки.

# 1 Анализ предметной области и формирование требований

## 1.1 Общая характеристика предметной области

Предметная область работы находится на пересечении анализа исполняемых файлов, системного программирования и инструментов информационной безопасности. PE-файл является контейнером, который связывает машинный код, таблицы импорта, данные о секциях, ресурсы и служебные каталоги. Для операционной системы такой файл является не просто набором байтов, а договором о том, как загрузить образ в память, какие адреса считать допустимыми и какие структуры использовать при выполнении.

В 64-разрядной Windows механизм обработки исключений тесно связан с данными раскрутки стека. Когда возникает исключение, системе необходимо пройти по стеку вызовов, определить границы текущей функции, восстановить сохраненные регистры и найти возможный обработчик. Для этого используются записи RUNTIME_FUNCTION и связанные структуры UNWIND_INFO [3], [4]. Если эти структуры повреждены, противоречивы или намеренно размещены необычным способом, то корректная интерпретация поведения программы становится сложнее.

На практике EH/Unwind-метаданные важны не только для штатной работы программы, но и для анализа защищенного или нестандартного кода. Упаковщики, виртуализаторы, средства обфускации и ручные модификации PE-образов могут менять привычное расположение служебных структур. При этом внешне файл может оставаться допустимым PE-образом, запускаться и даже выполнять полезную функцию. Поэтому аналитик нуждается в инструменте, который выделяет именно те особенности, которые трудно заметить при беглом просмотре.

В отличие от общего дизассемблирования, анализ EH/Unwind-слоя не требует восстановления всего графа управления программы. Он работает с ограниченным набором структур, но предъявляет высокие требования к проверке границ и адресов. Ошибка в преобразовании RVA в файловое

смещение, неверная интерпретация размера секции или отсутствие проверки переполнения может привести к ложному выводу. Поэтому ключевым свойством инструмента является безопасное чтение недоверенного файла.

Для данной работы важно разделять понятия подозрительности и доказанности. Поврежденная структура UNWIND_INFO или обработчик в необычной секции не доказывает вредоносность файла. Такие признаки являются основанием для дальнейшей проверки, поскольку похожие состояния могут возникать и в легитимных программах, собранных нестандартными компиляторами или защищенных коммерческими средствами. Следовательно, результат анализа должен быть объяснимым и воспроизводимым.

Вывод по подразделу: предметная область требует специализированного инструмента, который работает с низкоуровневыми структурами PE32+ AMD64, не запускает файл в статическом режиме, фиксирует причины ошибок и представляет результаты в форме, понятной для последующей ручной проверки.

### 1.2 EH/Unwind-метаданные PE32+ AMD64

В PE32+ AMD64 таблица исключений хранится в каталоге данных IMAGE_DIRECTORY_ENTRY_EXCEPTION. Для 64-разрядных образов этот каталог обычно указывает на секцию .pdata, содержащую массив записей RUNTIME_FUNCTION. Каждая запись описывает диапазон кода функции через начальный и конечный RVA, а также содержит ссылку на структуру UNWIND_INFO. Таким образом, таблица .pdata задает карту функций с точки зрения механизма обработки исключений Windows.

Структура UNWIND_INFO описывает пролог функции, операции сохранения регистров, размер фиксированного стекового кадра, наличие frame-регистра, обработчики исключений и цепочки записей. Эта информация необходима системе для корректной раскрутки стека без анализа машинных инструкций в момент исключения. В корректно

собранной программе данные должны соответствовать ABI и фактической организации функции [4].

Особенность AMD64 состоит в том, что таблица RUNTIME_FUNCTION описывает функции как полуоткрытые диапазоны адресов. Если BeginAddress не меньше EndAddress, если диапазоны пересекаются или выходят за границы образа, то карта функций становится неоднозначной. Даже если файл сохраняет возможность запуска, такая неоднозначность мешает отладке, профилированию и восстановлению логики программы.

Сами данные раскрутки также могут содержать ошибки. Например, количество unwind-кодов может не соответствовать длине структуры, операция может иметь недопустимый операнд, tail-часть может обрываться, а chained-запись может вести к циклу. Для ручного анализа такие случаи неудобны, потому что специалисту приходится проверять байтовую разметку, адреса и флаги одновременно.

В рамках работы EH/Unwind-слой рассматривается как самостоятельный объект анализа. Инструмент не пытается заменить дизассемблер, но дополняет его: сначала строится структурированная модель служебных данных, затем проверяются инварианты, после чего результат можно использовать в Ghidra, отладчике или другом средстве исследования.

Вывод по подразделу: .pdata и .xdata являются компактным, но информативным источником сведений о функциях и обработчиках исключений; их корректный анализ требует точной проверки адресов, размеров, флагов и связей между структурами.

### 1.3 Статический и динамический подходы к анализу

Статический анализ выполняется без запуска исследуемого файла. Его преимущество состоит в безопасности и воспроизводимости: один и тот же входной файл при одинаковых параметрах дает одинаковую модель структур. Такой режим подходит для недоверенных образцов, первичного

отбора и автоматической обработки больших наборов файлов. В данной работе статический режим является базовым и обязательным.

Ограничение статического анализа состоит в том, что он видит только записанные в файле структуры. Если программа во время выполнения регистрирует векторные обработчики исключений, динамические таблицы функций или меняет картину обработки исключений за счет API Windows, то это поведение нельзя полностью восстановить из байтов файла. Для таких случаев нужен динамический сбор событий.

Динамический анализ запускает исследуемую программу и наблюдает за ее поведением. В проекте он реализован через Windows Debug API: цель создается как отлаживаемый процесс, цикл обработки debug-событий фиксирует исключения, регистрации обработчиков и вызовы функций, связанных с динамическими таблицами функций [5]. Такой подход позволяет увидеть поведение, которое не отражено напрямую в .pdata и .xdata.

Динамический режим имеет более высокие риски. Он исполняет файл, поэтому неизвестные образцы необходимо запускать только в изолированной виртуальной машине. Кроме того, динамический результат зависит от входных параметров, окружения, таймаута, прав доступа и факта достижения нужного участка кода. Поэтому динамика не заменяет статику, а используется как дополнительный слой технической доказательности.

В рамках работы статическая и динамическая части разделены по ответственности. Статика строит модель файла и выявляет признаки, доступные без запуска. Динамика фиксирует наблюдаемые события выполнения и возвращает их в общий отчет. Финальный вывод формируется аккуратно: если динамическая сессия неполная или завершилась таймаутом, это не должно автоматически усиливать подозрительность.

Вывод по подразделу: корректная архитектура инструмента должна сочетать безопасный статический анализ с ограниченным динамическим наблюдением, явно показывая границы применимости каждого режима.

### 1.4 Анализ существующих средств

Для исследования PE-файлов применяются разные классы инструментов: PE-просмотрщики, дизассемблеры, отладчики, песочницы и универсальные средства анализа вредоносного программного обеспечения. Каждый класс полезен, но решает свою задачу. PE-просмотрщик хорошо показывает заголовки и секции, но редко объясняет взаимосвязь unwind-структур. Дизассемблер восстанавливает функции и код, но анализ EH/Unwind-слоя обычно требует ручной проверки.

Отладчик позволяет наблюдать исключения, точки останова и динамическое поведение, но результат отладочной сессии зависит от выбранного сценария запуска. Если аналитик не знает, где именно установить точки наблюдения, часть информации может быть пропущена. Кроме того, отладчик не формирует сам по себе машиночитаемый отчет по всем функциям из таблицы исключений.

Песочницы и поведенческие анализаторы ориентированы на общий профиль активности: файловые операции, сеть, процессы, реестр, подозрительные API-вызовы. Они полезны для первичного исследования потенциально вредоносного объекта, но не предназначены для детального объяснения структуры UNWIND_INFO или причин повреждения конкретной записи RUNTIME_FUNCTION.

Ghidra и аналогичные среды дают мощные возможности для обратной разработки. Однако в учебной и лабораторной задаче полезно иметь отдельный предварительный слой, который готовит таблицу функций, комментарии и предупреждения для импорта. Такой слой экономит время и делает исследование более воспроизводимым, поскольку одни и те же правила применяются до ручной работы специалиста.

Сравнение существующих подходов приведено в таблице 2. Оно показывает, что разработка отдельного инструмента оправдана не конкуренцией с большими средами, а узкой специализацией: автоматическим и объяснимым анализом EH/Unwind-метаданных.

Таблица 2 - Сравнение подходов к анализу PE-файлов

| Средство | Преимущество | Ограничение |
|---|---|---|
| PE-просмотрщик | быстро показывает заголовки и секции | не объясняет взаимосвязи EH/Unwind-структур |
| Дизассемблер | помогает изучать код и функции | требует ручной интерпретации unwind-данных |
| Отладчик | показывает поведение во время выполнения | не формирует полный отчет по .pdata/.xdata |
| Песочница | дает общий поведенческий профиль | не предназначена для детального разбора UNWIND_INFO |
| Разработанный инструмент | специализированно анализирует EH/Unwind-слой | не заменяет дизассемблер и песочницу |

### 1.5 Требования к разрабатываемому инструменту

На основании анализа предметной области и существующих средств сформированы требования к программному инструменту. Они разделены на функциональные и нефункциональные, поскольку часть требований описывает реализуемые возможности, а часть - ограничения безопасности, переносимости и проверяемости.

Таблица 3 - Основные требования к инструменту

| Тип | Требование |
|---|---|
| Функциональное | Разбор PE32+ AMD64, чтение .pdata и декодирование UNWIND_INFO |
| Функциональное | Поиск структурных аномалий и формирование объяснимых признаков |
| Функциональное | Динамический сбор событий исключений Windows x64 |
| Функциональное | Формирование JSON, HTML, CSV и Ghidra CSV |
| Нефункциональное | Безопасное чтение недоверенных файлов без запуска в статическом режиме |
| Нефункциональное | Сборка разными компиляторами и проверяемые отчеты |

Главным требованием является воспроизводимость результата. Если инструмент сообщает о признаке, в отчете должны быть указаны структура, адрес, семейство признака и причина вывода. Это позволяет специалисту проверить результат независимо от внутренней реализации.

Динамический режим не должен объявляться универсальным или кроссплатформенным. Его область применения ограничена Windows x64, потому что сбор событий основан на Windows Debug API и конкретных механизмах обработки исключений этой платформы.

Вывод по подразделу: требования фиксируют четыре ключевых блока проекта - парсинг, статический анализ, динамический анализ и отчеты; каждый блок должен иметь проверяемый результат и понятные ограничения.

### 1.6 Критерии корректности выявления аномалий

Для рассматриваемой задачи важно различать два близких, но не одинаковых понятия: обнаружение технического несоответствия и вывод о вредоносности. Разработанный инструмент решает первую задачу. Он не относит файл к вредоносным или безопасным автоматически, а выявляет такие свойства EH/Unwind-слоя, которые требуют внимания специалиста. Благодаря этому результат остается инженерно проверяемым и не превращается в непрозрачную классификацию.

Корректным признаком считается только такое наблюдение, которое может быть привязано к конкретной структуре входного файла. Например, если запись RUNTIME_FUNCTION выходит за допустимые границы, в отчете должны быть указаны адрес записи, проблемный диапазон и причина нарушения. Если обнаружена странная область размещения UNWIND_INFO, в отчете должна быть понятна секция, ее права доступа и связь с анализируемой записью. Без этой связи признак нельзя использовать как надежное основание для ручной проверки.

Вторым критерием является воспроизводимость. При одинаковом входном файле и одинаковых параметрах статический анализ должен возвращать одинаковый набор структур, признаков и отчетных полей. Это требование особенно важно для ВКР, потому что комиссия и нормоконтроль оценивают не только наличие программы, но и возможность проверить заявленный результат. Поэтому в проекте используются детерминированные правила обхода таблиц, фиксированные границы чтения и машинно проверяемая схема JSON-отчета.

Третьим критерием является устойчивость к поврежденному входу. PE-файл, переданный на анализ, не должен рассматриваться как доверенный. Даже если заголовок содержит некорректный размер, неверный RVA или

обрывающуюся таблицу, программа обязана завершить обработку управляемо: сформировать диагностическое сообщение, вернуть понятный статус и не выйти за границы буфера. Именно поэтому парсинг выделен в отдельный слой и опирается на безопасные функции чтения.

Четвертым критерием является разделение уровня серьезности и уровня уверенности. Серьезность отражает потенциальную опасность найденного несоответствия для последующего анализа, а уверенность показывает, насколько однозначно инструмент смог интерпретировать данные. Такая модель помогает избежать ложной категоричности: слабый признак с низкой уверенностью не должен подаваться так же, как грубое повреждение таблицы исключений с прямым указанием на неверный диапазон.

Вывод по подразделу: корректное выявление аномалий в данном проекте определяется не количеством найденных предупреждений, а их проверяемостью, воспроизводимостью, устойчивостью к поврежденным данным и ясной связью с конкретными EH/Unwind-структурами.

### 1.7 Роль объяснимости в инструменте анализа

При анализе исполняемых файлов особенно опасны выводы, которые невозможно проверить. Если инструмент сообщает только итоговый балл, специалисту приходится заново искать причину срабатывания вручную. Поэтому в разрабатываемом приложении отчет рассматривается как часть функциональности, а не как второстепенный вывод в конце работы. Отчет должен отвечать на вопросы: что найдено, где найдено, почему это считается подозрительным и что проверить дальше.

Объяснимость нужна и для снижения ложных срабатываний. В нормальном PE-файле могут встречаться нестандартные, но допустимые ситуации: особенности компилятора, оптимизации, системные библиотеки, защитные механизмы или нестандартная компоновка. Если инструмент явно показывает основание признака, специалист может отличить реальную структурную проблему от допустимого варианта сборки. Это особенно важно

для EH/Unwind-метаданных, потому что они тесно связаны с компилятором и ABI.

В проекте объяснимость реализуется через несколько уровней представления результата. Консольный вывод подходит для быстрого просмотра. JSON нужен для автоматической проверки и интеграции. HTML помогает просматривать результат человеком без отдельной среды анализа. CSV удобен для табличной обработки, а Ghidra CSV переносит часть сведений в инструмент обратной разработки. Такое разделение форматов позволяет использовать один и тот же результат в разных рабочих сценариях.

Отдельное значение имеет объяснение ограничений. Динамический режим не должен восприниматься как универсальная песочница: он выполняет файл, зависит от сценария запуска и доступен только на Windows x64. Поэтому в пояснительной записке и в отчетах важно прямо указывать границы применимости. Наличие ограничений не снижает ценность проекта, если эти ограничения сформулированы честно и связаны с выбранной предметной областью.

Вывод по подразделу: объяснимость является одним из основных требований к инструменту, потому что именно она превращает автоматический анализ в проверяемую помощь специалисту и делает результаты пригодными для защиты, повторной проверки и дальнейшего развития.

## 2 Проектирование и разработка инструмента

### 2.1 Выбор инструментов разработки

Для реализации выбран язык C. Такой выбор связан с характером задачи: инструмент работает с бинарными структурами, смещениями, адресами и фиксированными размерами полей. Язык C позволяет явно контролировать представление данных, не требует тяжелой среды выполнения и подходит для создания консольной утилиты, которую можно собрать разными компиляторами [13].

Проект использует CMake как систему сборки и Ninja как быстрый генератор сборочных файлов [14], [15]. Это позволяет фиксировать профили Debug, Release, Sanitizer и Coverage, а также проверять проект на разных компиляторах. Для Windows используются MinGW-w64 и MSVC, что снижает риск привязки к одному инструментальному окружению.

Статический режим не использует Windows-заголовки в базовом PE-парсере. Это важно, потому что анализ файла как набора байтов не должен зависеть от платформы запуска. Динамический режим, наоборот, имеет явную платформенную зависимость: он использует Windows Debug API и предназначен только для Windows x64. Такое разделение отражено в архитектуре и требованиях.

Для выходных данных выбран набор форматов: JSON, HTML, CSV и Ghidra CSV. JSON нужен для автоматической обработки и проверки схемой; HTML удобен для человека; CSV позволяет анализировать таблицу функций в табличном редакторе; Ghidra CSV служит интеграцией с инструментом обратной разработки. Наличие нескольких форматов позволяет использовать один результат в разных сценариях.

Выбор инструментов подчинен принципу минимальности. Проект не требует базы данных, серверной части или графической оболочки, потому что его базовый сценарий - консольный анализ файла и формирование отчетов. Это упрощает сборку, снижает число зависимостей и делает инструмент пригодным для лабораторного применения.

Вывод по подразделу: выбранный стек соответствует задаче низкоуровневого анализа, обеспечивает контроль над бинарным разбором, поддерживает проверку на разных компиляторах и оставляет динамическую часть строго ограниченной Windows x64.

### 2.2 Общая архитектура программного средства

Архитектура инструмента построена как последовательный конвейер обработки данных. На вход поступает PE-файл, затем выполняется безопасное чтение байтов, разбор заголовков, построение карты секций, чтение каталога исключений, декодирование unwind-структур, статическая оценка, дополнительный динамический сбор и формирование отчетов. Такая структура показана на рисунке 1.

Разделение на модули позволяет изолировать ответственность. Модуль util.c отвечает за безопасное чтение, строки, контрольные хеш-суммы и файловые операции. Модуль pe.c описывает работу с заголовками и секциями. Модуль unwind.c декодирует таблицы исключений и UNWIND_INFO. Модуль heuristics.c формирует статические признаки. Модуль dynamic_win.c собирает события выполнения. Модуль report.c сериализует результат.

Важным проектным решением является отсутствие единого глобального состояния. Основные структуры создаются, передаются по конвейеру и освобождаются вызывающим кодом. Это упрощает обработку ошибок: если разбор завершился раньше времени, программа может освободить уже созданные объекты и вернуть понятную причину отказа.

Конвейер также удобен для тестирования. Каждый модуль можно проверять отдельно: PE-парсер на корректных и поврежденных заголовках, unwind-декодер на синтетических структурах, эвристики на заранее размеченных наборах, отчеты на соответствие JSON-схеме, динамический сбор на контролируемых сценариях VEH, VCH, UEF и RtlAddFunctionTable.

Архитектура не пытается смешивать статические и динамические данные на ранних этапах. Сначала формируется статическая модель, затем

динамический блок получает ее как входные данные для установки точек наблюдения и сопоставления событий. Благодаря этому динамика остается дополнительным режимом, а не обязательной частью анализа.

Вывод по подразделу: модульная конвейерная архитектура соответствует требованиям воспроизводимости, тестируемости и безопасной обработки недоверенных PE-файлов.

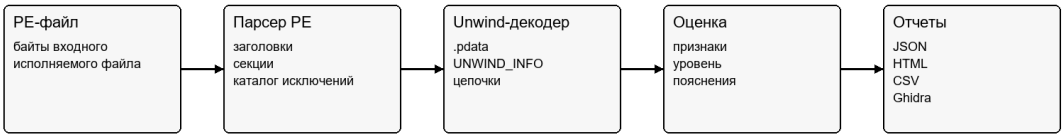



Рисунок 1 - Конвейер обработки данных в инструменте

Таблица 4 - Модули программного средства

| Модуль | Назначение |
|---|---|
| util.c | безопасное чтение, строки, файлы, хеш-суммы |
| pe.c | PE32+ AMD64, секции, RVA, каталог исключений |
| unwind.c | RUNTIME_FUNCTION, UNWIND_INFO, цепочки, ABI-диагностика |
| heuristics.c | признаки, балл, уверенность, правила ослабления |
| dynamic_win.c | Windows Debug API, события исключений, точки останова |
| report.c | консольный вывод, JSON, HTML, CSV, Ghidra CSV |
| main.c | CLI, конвейер, обработка ошибок и освобождение ресурсов |

### 2.3 Модель входных и выходных данных

Входными данными является путь к PE-файлу и набор параметров командной строки. Основные параметры включают вывод таблицы функций, декодирование операций раскрутки, запуск эвристической оценки, включение строгих ABI-диагностик, динамический режим, таймаут, число шагов трассировки и пути к отчетам. Пользователь может одновременно запросить несколько форматов вывода.

Внутренняя модель начинается со структуры PeImage. Она хранит базовый адрес образа, размеры, параметры выравнивания, каталог исключений, таблицу секций и счетчики нарушений. Наличие счетчиков

прямо в модели упрощает дальнейшую эвристическую оценку: результаты структурной проверки не теряются как текстовые сообщения, а становятся числовыми признаками.

Структура RuntimeFunction содержит три поля: начало диапазона функции, конец диапазона и адрес UNWIND_INFO. Эта простая модель отражает формат таблицы .pdata и позволяет отдельно проверять сортировку, пересечения, выходы за границы и корректность ссылок на данные раскрутки.

Структура UnwindSummary является результатом декодирования UNWIND_INFO. Она содержит версию, флаги, размер пролога, число unwind-кодов, frame-регистр, сведения об обработчике, цепочке, фиксированном размере стекового кадра, сохраненных регистрах и битовой маске аномалий. Такая форма удобна для отчетов и для эвристической оценки.

Выходная JSON-модель фиксируется схемой. В ней присутствуют метаданные отчета, сведения об образе, список секций, результаты проверки PE, блок статической оценки, блок динамической проверки, итоговая оценка и массив функций. Схема запрещает лишние поля и задает типы данных, что уменьшает риск несовместимости между версиями отчета [16].

Вывод по подразделу: модель данных сохраняет связь между низкоуровневым байтовым разбором и высокоуровневым отчетом, что делает результат пригодным и для человека, и для автоматической проверки.

Таблица 5 - Основные внутренние структуры данных

| Структура | Содержание |
|---|---|
| PeImage | карта PE-образа, секции, каталог исключений, структурные нарушения |
| RuntimeFunction | диапазон функции и ссылка на данные раскрутки |
| UnwindSummary | декодированная модель UNWIND_INFO и обработчиков |
| HeuristicReport | признаки, балл, уверенность, доказательность |
| DynamicReport | исключения, обработчики, runtime-таблицы, счетчики полноты |

### 2.4 Алгоритм парсинга PE32+ AMD64

Парсинг начинается с чтения файла в буфер и проверки минимальных размеров. Затем анализируется DOS-заголовок, значение e_lfanew, сигнатура PE, файловый заголовок и optional header. На этом этапе инструмент проверяет, что файл действительно является PE32+ образом для AMD64. Образы другой архитектуры или другого формата отклоняются с диагностикой.

После проверки заголовков строится таблица секций. Для каждой секции сохраняются имя, виртуальный адрес, виртуальный размер, файловое смещение, размер raw-данных и характеристики. Одновременно выполняются проверки на выход raw-диапазона за размер файла, выход виртуального диапазона за размер образа, пересечения и нарушения выравнивания.

Ключевой операцией является преобразование RVA в файловое смещение. Для корректного преобразования необходимо найти секцию, в виртуальный диапазон которой попадает адрес, и затем вычислить смещение относительно raw-данных. Ошибка в этом месте приводит к чтению неправильных байтов, поэтому функция преобразования возвращает явный признак успеха или отказа.

Каталог исключений анализируется как отдельный объект. Инструмент проверяет, что его RVA и размер находятся в допустимом диапазоне, а размер кратен длине записи RUNTIME_FUNCTION. Если каталог отсутствует или поврежден, программа не должна аварийно завершаться; она формирует диагностический результат, который попадает в отчет.

Особое внимание уделено переполнениям и неполному чтению. Все операции чтения используют функции, которые проверяют границы буфера. Такой подход важен для анализа недоверенных файлов: поврежденный PE-образ не должен приводить к чтению за пределами памяти инструмента.

Вывод по подразделу: алгоритм парсинга PE32+ AMD64 построен как последовательная проверка доверенных предположений; каждое

преобразование адреса и каждое чтение структуры сопровождаются проверкой границ.

### 2.5 Декодирование RUNTIME_FUNCTION и UNWIND_INFO

После чтения каталога исключений инструмент получает массив записей RUNTIME_FUNCTION. Каждая запись анализируется как диапазон функции и ссылка на unwind-структуру. На этом этапе проверяются нулевые и инвертированные диапазоны, выходы за границы образа, пересечения, повторяющиеся начальные адреса и несортированность таблицы.

Декодирование UNWIND_INFO начинается с чтения первого байта, содержащего версию и флаги. Затем извлекаются размер пролога, число unwind-кодов, frame-регистр и frame-offset. После этого декодируются операции восстановления стека. Инструмент оценивает фиксированный размер стекового кадра, сохраненные регистры и возможные ABI-диагностики.

Если структура содержит флаги EHANDLER или UHANDLER, после массива unwind-кодов ожидается RVA обработчика. Если установлен CHAININFO, tail-часть содержит связанную запись RUNTIME_FUNCTION. Инструмент поддерживает цепочки с ограничением глубины, чтобы избежать бесконечных переходов при поврежденных или специально созданных данных.

Формальная проверка ABI вынесена в отдельный режим. Это важно, потому что строгие предупреждения не всегда означают практическую проблему: реальные компиляторы и инструменты могут генерировать допустимые с точки зрения выполнения, но неидеальные с точки зрения формального описания записи. Поэтому строгие ABI-диагностики видны в отчете, но не должны автоматически увеличивать подозрительность.

Результатом декодирования является не только текст операций, но и нормализованная структура UnwindSummary. Это позволяет формировать таблицы, JSON-отчет, HTML-представление и признаки для эвристической модели из одного источника данных.

Вывод по подразделу: декодер unwind-структур выполняет не просто печать полей, а строит проверяемую модель функции, обработчика и правил восстановления стека.

### 2.6 Эвристическая модель оценки аномалий

Эвристическая модель нужна для того, чтобы перевести набор низкоуровневых нарушений в понятный аналитический вывод. При этом модель не должна выдавать статистическую вероятность вредоносности. В работе оценка трактуется как инженерная шкала подозрительности, помогающая расставить приоритеты ручной проверки.

Признаки объединены в семейства: целостность таблицы, целостность UNWIND_INFO, размещение метаданных, повторное использование unwind-данных и доминирование обработчиков. Такое разделение помогает объяснить причину вывода. Например, повреждение диапазонов функций и размещение обработчика в необычной секции являются разными типами сигналов и должны отображаться отдельно.

Каждый признак имеет код, семейство, серьезность, сообщение, число затронутых объектов и представительный адрес. Это делает отчет проверяемым: аналитик может перейти к конкретной структуре, а не видеть только итоговый балл. Максимальное число признаков ограничено, чтобы отчет оставался читаемым даже на крупных файлах.

Модель также содержит правила ослабления уверенности. Если найден один слабый признак, если выборка функций мала или если сигнал относится только к одному семейству, уверенность снижается. Это снижает риск переоценки изолированных особенностей, которые могут иметь легитимное объяснение.

Итоговая оценка состоит из балла, уровня доказательности, уровня уверенности, вердикта и признака необходимости ручной проверки. Важная особенность состоит в том, что уровень уверенности не является вероятностью вредоносности. Он показывает степень устойчивости интерпретации при имеющихся данных.

Вывод по подразделу: эвристическая модель делает анализ объяснимым и воспроизводимым, поскольку каждое повышение или ослабление оценки связано с конкретным техническим признаком.

### 2.7 Динамический сбор событий Windows

Динамический режим реализован как отдельный Windows-only модуль. Он запускает исследуемый исполняемый файл под отладчиком и обрабатывает события отладочного цикла. Главная цель режима - не полная трассировка программы, а сбор ограниченного набора событий, связанных с обработкой исключений и динамическими таблицами функций.

Модуль наблюдает first-chance и second-chance исключения, регистрации VEH и VCH, установку UEF, вызовы RtlAddFunctionTable и RtlInstallFunctionTableCallback, а также попадания в обработчики, которые были обнаружены статическим анализом. Для части событий сохраняются аргументы и адреса, что позволяет сопоставить их со статическими структурами [6]-[10].

Для устойчивости динамической проверки введены счетчики полноты инструментации. Отчет показывает количество попыток установки breakpoints, ошибки установки, потери из-за ограничения емкости, проблемы повторной установки точек и число отброшенных событий. Благодаря этому неполный динамический запуск не маскируется под чистый результат.

Динамический анализ ограничивается таймаутом. Если цель не завершилась естественно, отчет фиксирует timed_out и сохраняет уже собранные события. Такая сессия не должна автоматически усиливать итоговый вывод, потому что отсутствие события в неполной трассе не означает отсутствие соответствующего поведения в программе.

Безопасность динамического режима является отдельным требованием. Поскольку файл исполняется, запуск неизвестных объектов возможен только в изолированной виртуальной машине. Статический режим остается основным безопасным способом анализа, а динамический используется

тогда, когда требуется дополнительное подтверждение или демонстрация runtime-механизмов.

Вывод по подразделу: динамический сбор расширяет информативность отчета, но сохраняет явные ограничения по платформе, безопасности, таймауту и полноте наблюдения.

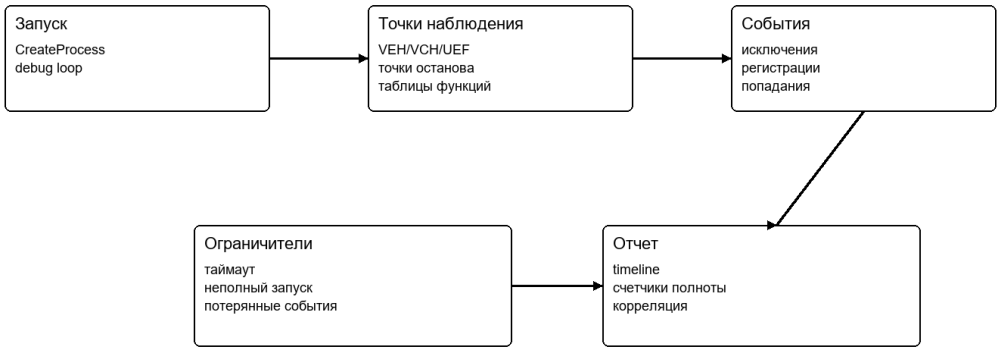



Рисунок 2 - Логика динамического цикла наблюдения

### 2.8 Формирование отчетов и интеграция с Ghidra

Отчеты являются важной частью проекта, потому что результат анализа должен быть пригоден для повторной проверки. Консольный вывод удобен для быстрого запуска, но недостаточен для хранения и автоматизации. Поэтому проект формирует несколько дополнительных форматов, каждый из которых решает отдельную задачу.

JSON-отчет содержит полную модель анализа: метаданные, сведения о файле, секции, результаты PE-проверки, функции, unwind-структуры, признаки, динамические события и итоговую оценку. Для него задана JSON Schema, что позволяет автоматически проверять структуру и типы полей. Это особенно важно при регрессионном тестировании и обработке больших наборов файлов.

HTML-отчет предназначен для человека. Он является самодостаточным файлом без сетевых ресурсов, внешних скриптов и шрифтов. Такой отчет можно открыть локально и использовать на защите или в лаборатории. CSV-отчет, наоборот, нужен для табличной обработки: он

содержит плоский список функций и ключевые поля, которые удобно фильтровать.

Ghidra CSV является специализированным форматом интеграции. Импортер создает или обновляет функции, добавляет комментарии, предупреждения и закладки. При этом он должен работать аккуратно: не затирать пользовательские имена, проверять диапазоны адресов и пропускать конфликтующие функции. Такой подход делает инструмент частью практического процесса обратной разработки.

Запись файлов реализована с учетом отказов: отчет формируется во временный соседний файл, после чего выполняется атомарная замена. Если процесс прерывается, ранее существующий отчет не должен быть поврежден. Это требование особенно важно при пакетном анализе, где один сбой не должен уничтожать предыдущие результаты.

Вывод по подразделу: система отчетов связывает низкоуровневый анализ с практическим использованием результата, обеспечивая читаемость, машиночитаемость и интеграцию с внешним инструментом.

Таблица 6 - Форматы выходных данных

| **Формат** | **Назначение** |
|---|---|
| JSON | полная машиночитаемая модель результата |
| HTML | самодостаточный отчет для просмотра человеком |
| CSV | плоская таблица функций и unwind-полей |
| Ghidra CSV | диапазоны функций и комментарии для импорта |




Рисунок 3 - Сопоставление статического и динамического анализа

### 2.9 Границы доверия и безопасность

В проекте выделены несколько границ доверия. Первая граница проходит между входным файлом и статическим анализатором. Исследуемый PE-файл считается недоверенным набором байтов, поэтому каждое чтение должно проверяться. Инструмент не должен доверять размерам, адресам, количеству секций или длине таблицы исключений только потому, что они записаны в заголовке.

Вторая граница связана с динамическим режимом. Здесь недоверенный файл уже исполняется, поэтому риск принципиально выше. Такой режим нельзя использовать как безопасный просмотрщик. В документации и в пояснительной записке это ограничение фиксируется явно: динамику следует запускать только в изолированной среде, где возможное вредоносное поведение не повредит основной системе.

Третья граница связана с отчетами. JSON и HTML могут содержать строки, пришедшие из путей, метаданных или ошибок. Поэтому сериализация должна экранировать данные, не вставлять неэкранированный HTML и не обращаться к сетевым ресурсам. Это защищает аналитика от ситуации, когда исследуемый путь или имя файла влияет на отображение отчета.

Четвертая граница связана с импортом в Ghidra. CSV-файл является внешним вводом для скрипта, поэтому импортер должен проверять набор колонок, адресные диапазоны и конфликты с уже существующими функциями. Он не должен безусловно доверять данным даже тогда, когда они созданы собственным анализатором.

Такой подход соответствует инженерному принципу fail closed: при невозможности доказать корректность структуры инструмент должен возвращать диагностируемую ошибку, а не продолжать анализ на сомнительных предположениях. Это особенно важно для ВКР, где требуется показать не только работоспособность, но и качество проектных решений.

Вывод по подразделу: проект учитывает границы доверия между входным файлом, динамическим запуском, отчетами и внешними интеграциями, что повышает надежность и применимость инструмента.

### 2.10 Обработка ошибок и отказоустойчивость

Проектирование отказоустойчивости начинается с признания того, что входной файл может быть частично поврежденным или намеренно сконструированным для проверки границ парсера. Поэтому программа не должна доверять значениям из заголовков PE без дополнительной проверки. Размеры структур, смещения, RVA, число секций и каталог исключений рассматриваются как данные, требующие валидации перед каждым чтением.

В модуле безопасного чтения реализуется общий принцип: операция возвращает успех только в том случае, если весь требуемый диапазон находится внутри загруженного буфера. Такой подход предотвращает типичную ошибку низкоуровневых парсеров, когда корректность проверяется только для начального адреса, но не для конца диапазона. Для анализа исполняемых файлов это критично, потому что многие структуры имеют переменный размер и могут ссылаться друг на друга.

Преобразование RVA в файловое смещение выполняется через карту секций. Прямая арифметика без учета VirtualAddress, VirtualSize, PointerToRawData и SizeOfRawData может дать неправильный результат, особенно для файлов с нетипичной компоновкой. Поэтому проект проверяет принадлежность адреса секции и только после этого позволяет читать соответствующие байты из файла. Если адрес не сопоставляется с файловым диапазоном, формируется диагностируемое состояние.

Отдельно обрабатываются ситуации, когда таблица исключений присутствует формально, но ее размер не кратен размеру записи RUNTIME_FUNCTION, содержит пересекающиеся диапазоны или указывает на отсутствующие UNWIND_INFO. Такие случаи не должны приводить к аварийному завершению. Они переводятся в признаки и сообщения отчета, потому что сами по себе являются полезным результатом анализа.

Для динамического режима отказоустойчивость означает другое: программа должна корректно различать завершение цели, таймаут, отсутствие прав, невозможность запустить библиотеку как исполняемый файл и штатное завершение наблюдения. Эти состояния нельзя смешивать в одну ошибку, иначе результаты динамики будут непригодны для анализа. Поэтому отчет динамического режима фиксирует не только события, но и полноту наблюдения.

Вывод по подразделу: отказоустойчивость проекта строится на проверке каждого доверительного перехода: от байтов файла к PE-структурам, от RVA к файловым смещениям, от статического отчета к динамическому запуску и от внутренних структур к пользовательским форматам вывода.

### 2.11 Связь модулей с требованиями

Архитектура проекта построена так, чтобы каждое требование имело понятное место реализации. Требование безопасного парсинга относится к модулям util.c и pe.c. Требование интерпретации EH/Unwind-метаданных относится к unwind.c. Требование выявления признаков относится к heuristics.c. Требование динамического наблюдения относится к dynamic_win.c. Требование отчетности относится к report.c. Такая связь упрощает проверку проекта и объяснение его структуры комиссии.

Модуль main.c выполняет роль координатора. Он разбирает параметры командной строки, определяет требуемые режимы, запускает статический анализ, при необходимости подключает динамический сбор и передает итоговые структуры в подсистему отчетности. Важно, что main.c не должен содержать сложную предметную логику. Его задача состоит в управлении сценарием, а не в декодировании PE или подсчете эвристик.

Модуль pe.c отвечает за базовую модель исполняемого файла. Он не должен знать о том, какие признаки будут считаться подозрительными. Его ответственность ограничена корректным чтением заголовков, секций, каталогов данных и преобразованием адресов. Такое разделение снижает

риск, что эвристические решения начнут влиять на низкоуровневую корректность парсинга.

Модуль unwind.c является предметным ядром проекта. Он переводит записи таблицы исключений и UNWIND_INFO в более удобную внутреннюю модель. Именно здесь учитываются цепочки, обработчики, размеры кодов раскрутки и особенности x64 ABI. Ошибки этого модуля напрямую влияют на качество анализа, поэтому для него особенно важны граничные тесты и синтетические образцы.

Модуль heuristics.c не должен скрывать причину решения. Его задача состоит в том, чтобы превратить набор структурных фактов в список признаков, балл, уровень уверенности и рекомендацию по ручной проверке. Если в будущем набор признаков будет расширяться, этот модуль можно развивать без изменения парсера и форматов входных данных.

Модуль report.c отделяет представление результата от логики анализа. Это важно для сопровождения: добавление нового формата вывода не должно менять алгоритмы обнаружения признаков. Кроме того, отдельная подсистема отчетности позволяет централизованно решать вопросы экранирования HTML, корректности CSV и соответствия JSON-схеме.

Вывод по подразделу: модульная структура проекта соответствует четырем основным блокам работы - парсингу, статическому анализу, динамическому анализу и формированию отчетов, а границы модулей позволяют проверять каждую часть отдельно.

## 3 Проверка работы и оценка результатов

### 3.1 Структура реализации

Реализация разделена на исходные файлы production-ядра, заголовочные файлы, тестовую инфраструктуру, документацию и отчеты проверки. Каталог src содержит основные модули: util.c, pe.c, unwind.c, heuristics.c, dynamic_win.c, report.c, app_info.c и main.c. Каталог include содержит соответствующие публичные структуры и функции.

Файл main.c отвечает за жизненный цикл запуска: разбор параметров командной строки, чтение входного файла, последовательный вызов модулей, выбор режимов, обработку ошибок и освобождение ресурсов. Такая концентрация orchestration-кода упрощает понимание программы: доменная логика остается в отдельных модулях, а main.c связывает их в сценарий выполнения.

Модуль util.c предоставляет низкоуровневые операции, от которых зависит безопасность всего проекта: чтение little-endian чисел, проверка границ, работа со строковым буфером, чтение файла и вычисление контрольной хеш-суммы. Вынесение этих операций в отдельный модуль снижает вероятность дублирования небезопасного кода.

Модуль pe.c реализует проверку PE32+ AMD64 и построение карты секций. Он не зависит от Windows SDK и работает с собственными определениями нужных констант. Это позволяет выполнять статический анализ на разных платформах, если доступны компилятор C и файловая система.

Модуль unwind.c реализует чтение RUNTIME_FUNCTION и декодирование UNWIND_INFO. Он формирует диагностические признаки, а также поддерживает параметры follow_chain, max_chain_depth и strict_abi. Модуль heuristics.c принимает уже нормализованные данные и не занимается байтовым чтением, что разделяет уровни ответственности.

Вывод по подразделу: структура реализации соответствует проектной архитектуре, так как каждый модуль имеет ограниченную область ответственности и может быть проверен отдельно.

Таблица 7 - Каталоги проекта

| Каталог | Назначение |
|---|---|
| src | исходные файлы production-ядра |
| include | публичные заголовки и структуры |
| tests/unit | модульные тесты |
| tests/samples | воспроизводимые PE-образцы |
| tests/fuzz | точка входа для fuzz-проверки парсера |
| docs | архитектура, схема отчета, модель признаков |
| reports | результаты финальной проверки |
| ghidra | скрипт импорта CSV в Ghidra |

### 3.2 Пользовательский сценарий работы

Базовый пользовательский сценарий начинается с запуска анализатора с путем к PE-файлу. Если дополнительные параметры не указаны, инструмент выполняет статический разбор и выводит краткую информацию о функциях и unwind-структурах. Для более подробного исследования пользователь включает вывод операций, эвристическую оценку и запись отчетов.

На практике один запуск может сформировать сразу несколько результатов. Например, JSON используется для автоматической проверки, HTML - для чтения, CSV - для таблиц, а Ghidra CSV - для импорта в среду обратной разработки. Такой подход уменьшает число повторных запусков и снижает риск расхождения данных между отчетами.

Если включается динамический режим, пользователь задает таймаут и, при необходимости, аргументы целевой программы после разделителя. Инструмент запускает цель под отладчиком Windows и фиксирует события. В отчете обязательно сохраняется информация о том, завершилась ли цель, был ли таймаут, сколько событий собрано и были ли потери.

Для неизвестных файлов сценарий должен начинаться только со статического режима. Динамический запуск допустим после подготовки изолированной среды. Это ограничение является частью пользовательских требований и должно быть понятно отражено в документации, потому что неправильное применение динамического режима может создать риск для основной системы.

Пользовательский сценарий ориентирован на специалиста, которому нужен не абстрактный итог, а проверяемые данные. Поэтому отчет содержит адреса, секции, причины parse-fail, список признаков и рекомендуемые шаги ручной проверки. Инструмент не требует от пользователя доверять итоговому score как черному ящику.

Вывод по подразделу: сценарий работы поддерживает как быстрый одиночный запуск, так и воспроизводимый лабораторный анализ с несколькими форматами отчетов.

### 3.3 Методика тестирования

Проверка инструмента построена на нескольких уровнях. Первый уровень - модульные тесты, которые проверяют отдельные функции чтения, парсинга, эвристик и отчетов. Второй уровень - регрессионные PE-образцы, в которых заранее известны корректные и аномальные случаи. Третий уровень - проверка реальных обычных PE-файлов. Четвертый уровень - динамические сценарии Windows.

Модульные тесты важны для защиты от локальных ошибок. Они позволяют проверить граничные случаи: короткий файл, неверные размеры, выход RVA за пределы секции, некорректную длину таблицы, поврежденный tail UNWIND_INFO, ограничения глубины цепочек и корректность JSON-вывода. Такие ошибки трудно ловить только ручным запуском.

Синтетические PE-образцы используются для проверки известных аномалий. Они позволяют задать конкретное нарушение и убедиться, что инструмент находит именно его. Это необходимо, потому что на реальных файлах не всегда известно, является ли необычная структура ошибкой, особенностью компилятора или результатом защиты.

Реальный набор обычных PE-файлов используется для оценки шумности эвристик. Если инструмент постоянно повышает score на нормальных файлах, он непригоден для triage. В проверке проекта обычные PE-файлы должны давать валидные отчеты и не создавать ложные сильные сигналы.

Динамические сценарии проверяют отдельные механизмы Windows: регистрацию VEH, VCH, UEF и добавление runtime function table. Такие тесты нужны, потому что динамический режим зависит от событий операционной системы и не может быть полностью проверен статическими образцами.

Вывод по подразделу: многоуровневая методика тестирования проверяет не только факт запуска программы, но и корректность отдельных алгоритмов, отчетов, эвристик и динамических наблюдений.

Таблица 8 - Уровни проверки программного средства

| Уровень | Назначение |
|---|---|
| Модульные тесты | проверка отдельных функций и граничных случаев |
| Синтетический набор | проверка известных аномалий |
| Обычные PE-файлы | оценка ложных срабатываний |
| Динамические smoke-тесты | проверка VEH/VCH/UEF/RtlFunctionTable |
| Полный динамический набор | проверка устойчивости Windows-сбора |

### 3.4 Результаты статической проверки

Статическая проверка включала два типа наборов: синтетический размеченный набор и набор обычных реальных PE-файлов. В синтетическом наборе присутствовали корректные файлы и файлы с заранее внесенными нарушениями EH/Unwind-структур. Это позволило проверить, что признаки поднимаются в ожидаемых ситуациях.

По результатам проверки синтетического набора все 14 отчетов были сформированы и прошли проверку. Обычные контрольные образцы получили нулевую оценку подозрительности, а аномальные образцы были подняты эвристиками с разными уровнями. Такой результат важен, потому что показывает способность инструмента различать нормальную структуру и контролируемые нарушения.

Реальный обычный набор включал 106 PE-файлов. Все 106 отчетов были валидны, суммарно разобрано 174 795 runtime-функций. Ненулевых эвристических оценок на этом наборе не зафиксировано. Это подтверждает,

что модель не дает массовых ложных срабатываний на проверенном наборе обычных файлов.

Особое значение имеет проверка JSON-схемы и семантических инвариантов. Схема подтверждает наличие обязательных полей и корректность типов, а семантические проверки проверяют связи между полями: размеры функций, счетчики, согласованность итоговой оценки и факта найденных признаков.

Статическая проверка также подтвердила корректность обработки кириллических путей в JSON и HTML-отчетах. Для Windows-проектов это практический момент: пути пользователя и папки на рабочем столе могут содержать русские символы, и отчет не должен терять их при сериализации.

Вывод по подразделу: результаты статической проверки подтверждают корректность парсинга, устойчивость отчетов и умеренность эвристической модели на проверенном наборе обычных PE-файлов.

### 3.5 Результаты динамической проверки

Динамическая проверка проводилась на контролируемых сценариях и полном Windows-наборе. Контролируемые сценарии проверяли VEH, VCH, UEF и RtlFunctionTable. Все четыре сценария прошли проверку, отчеты были валидны, потерянных событий не зафиксировано.

Полный динамический набор включал 124 входа. Все 124 отчета прошли проверку схемы и семантических инвариантов. Из них 86 debug-сессий были запущены, 54 цели завершились, 32 сессии завершились контролируемым таймаутом, 8 целей требовали повышенных прав, 30 DLL/PYD-файлов были ожидаемо отклонены CreateProcess как неподдерживаемые для прямого запуска.

В полном динамическом наборе зафиксированы 344 исключения, 136 попаданий в unwind-обработчики, активность VEH/VCH/UEF и вызовы функций динамических таблиц. Важно, что неожиданных динамических отказов и потерянных событий в проверочном наборе не зафиксировано. Это

подтверждает не абсолютную полноту трассировки, а устойчивость реализованного наблюдателя в заявленных границах.

Результаты elevation_required и unsupported_library_launch не считаются ошибками инструмента. Первый случай связан с тем, что некоторые цели требуют повышенных прав для отладки, второй - с невозможностью запускать DLL/PYD напрямую через CreateProcess. Оба состояния должны явно фиксироваться в отчете, чтобы пользователь не принимал их за потерю данных.

Динамическая проверка показывает, что режим выполнения полезен как дополнительный слой. Он не заменяет статический анализ, потому что зависит от сценария запуска, но позволяет подтвердить наличие runtime-механизмов, которые не видны или не полностью видны в статическом PE-образе.

Вывод по подразделу: динамический модуль корректно фиксирует заявленные события Windows, отражает неполные сценарии как отдельные состояния и не скрывает ограничения выполнения.

Таблица 9 - Результаты динамической проверки

| Показатель | Значение |
|---|---|
| Контролируемые динамические сценарии | 4 из 4 прошли проверку |
| Полный динамический набор | 124 из 124 валидных отчетов |
| Запущенные debug-сессии | 86 |
| Завершившиеся цели | 54 |
| Контролируемые таймауты | 32 |
| Неожиданные динамические отказы | 0 |
| Потерянные события | 0 |

### 3.6 Оценка соответствия требованиям

Функциональные требования к парсингу выполнены: инструмент читает PE32+ AMD64, строит карту секций, проверяет каталог исключений, извлекает записи RUNTIME_FUNCTION и декодирует UNWIND_INFO. При ошибках формируется диагностируемое состояние, а не аварийное завершение без пояснения.

Требования к статическому анализу выполнены за счет проверки целостности таблицы функций, корректности unwind-структур, размещения метаданных и повторного использования служебных записей. Эвристическая модель формирует не только итоговый score, но и список конкретных признаков с семейством, серьезностью и представительным адресом.

Требования к динамическому анализу выполнены в заявленных границах Windows x64. Инструмент собирает исключения, регистрации обработчиков, попадания в обработчики и события динамических таблиц функций. При этом динамический режим явно помечен как исполняющий файл и требующий изоляции для неизвестных образцов.

Требования к отчетности выполнены через JSON, HTML, CSV и Ghidra CSV. JSON проверяется схемой, HTML является самодостаточным, CSV пригоден для табличной обработки, а Ghidra CSV поддерживает перенос функций и комментариев в среду обратной разработки.

Нефункциональные требования также учитываются: код собирается MinGW-w64 и MSVC, статический режим не зависит от Windows API, динамический режим ограничен платформой Windows x64, отчеты создаются атомарно, а тестовая инфраструктура фиксирует регрессионные сценарии.

Вывод по подразделу: реализованное программное средство соответствует сформулированным требованиям и имеет проверочную базу, достаточную для демонстрации результатов ВКР.

### 3.7 Ограничения разработанного инструмента

Инструмент поддерживает PE32+ AMD64. Форматы PE32/x86, ARM64, ELF и Mach-O не входят в область работы. Такое ограничение не является недостатком реализации, а отражает выбранную тему ВКР: анализ EH/Unwind-метаданных Windows x64 имеет собственные структуры и правила, отличающиеся от других платформ.

Статический режим не выполняет дизассемблирование .text и не сравнивает каждую unwind-операцию с фактическими машинными инструкциями пролога. Поэтому он проверяет структурную корректность и

явные аномалии, но не доказывает полное соответствие unwind-данных машинному коду.

Динамический режим не является песочницей вредоносного программного обеспечения. Он не собирает полную трассу всех API-вызовов, файловых операций, сетевой активности или изменения реестра. Его задача уже: наблюдать события, связанные с исключениями и динамическими таблицами функций.

Результат динамики зависит от сценария запуска. Если нужный код не был достигнут, соответствующее событие может не появиться. Поэтому отсутствие runtime-сигнала не должно рассматриваться как доказательство отсутствия механизма в программе. В отчете это учитывается через состояния таймаута, полноты инструментации и отсутствие корреляции.

Эвристическая оценка не является классификатором вредоносности. Она помогает выделить файлы и структуры для ручной проверки. Такое ограничение важно явно проговаривать на защите, чтобы комиссия видела инженерную аккуратность и понимание границ разработанного решения.

Вывод по подразделу: ограничения инструмента определены заранее и не противоречат цели работы, поскольку проект решает специализированную задачу анализа EH/Unwind-слоя, а не универсальную задачу поведенческой классификации программ.

### 3.8 Практическое применение и развитие

Практический сценарий применения начинается с безопасного статического запуска на исследуемом файле. После получения JSON и HTML-отчета аналитик просматривает общую оценку, список признаков, секции, функции и причины ошибок. Если выявлены признаки, требующие ручной проверки, отчет предлагает адреса и шаги дальнейшего анализа.

Для более глубокого исследования можно сформировать Ghidra CSV и импортировать функции, комментарии и предупреждения в Ghidra. Это сокращает время подготовки проекта обратной разработки: часть служебной

информации уже перенесена в среду, а специалист может сосредоточиться на проверке конкретных участков.

В учебном процессе инструмент полезен как демонстрация связи между форматом PE, механизмом исключений Windows, статическим анализом и динамическим наблюдением. Студент может увидеть, что служебные структуры файла имеют практическое значение и что корректная обработка недоверенных данных требует инженерной дисциплины.

Направления развития включают поддержку новых архитектур, сопоставление unwind-данных с дизассемблированным прологом, расширение корреляционного слоя, графическое представление отчетов, интеграцию с песочницами и более подробное объяснение причин ослабления уверенности.

При развитии проекта важно сохранять принцип объяснимости. Добавление новых признаков должно сопровождаться документацией, тестами и обновлением схемы отчета. Иначе инструмент быстро станет непрозрачным, а его результаты будет трудно защищать и воспроизводить.

Вывод по подразделу: разработанное средство пригодно для практического лабораторного применения и имеет понятные направления дальнейшего развития без изменения базовой архитектуры.

### 3.9 Интерпретация результатов анализа

Результаты инструмента следует интерпретировать как техническую картину EH/Unwind-слоя, а не как окончательный экспертный приговор. Высокий балл означает, что в служебных структурах обнаружены несоответствия, которые требуют внимания. Низкий балл означает отсутствие выявленных признаков в рамках реализованной модели, но не гарантирует отсутствие других видов проблем в программе.

Для статического анализа главным результатом является карта структур: секции, таблица RUNTIME_FUNCTION, UNWIND_INFO, обработчики, цепочки и диагностические состояния. Если эта карта построена корректно, специалист получает основу для дальнейшей работы в

дизассемблере или отладчике. Если карта строится с ошибками из-за поврежденного файла, сами ошибки становятся частью результата и помогают локализовать проблемную область.

Для динамического анализа главным результатом является не только список событий, но и информация о полноте наблюдения. Например, завершение по таймауту не равно ошибке инструмента, если таймаут был ожидаемым ограничением сценария. Отсутствие обработчика в динамике не опровергает статический признак, потому что нужный участок кода мог не выполниться. Поэтому динамика усиливает выводы, но не заменяет статическую проверку.

Отчеты нужно читать от общего к частному. Сначала оценивается статус разбора и итоговый вердикт. Затем рассматриваются признаки с наибольшей серьезностью. После этого проверяются адреса, секции и типы структур, на которые указывает отчет. Такой порядок позволяет избежать ситуации, когда внимание сразу уходит в низкоуровневые детали, а общая картина файла остается неясной.

Особую роль играет Ghidra CSV. Он не является отдельным анализатором, а служит мостом между автоматическим разбором и ручной обратной разработкой. Перенос функций и комментариев в Ghidra помогает специалисту быстрее сопоставить найденные признаки с машинным кодом, но окончательная интерпретация все равно остается за человеком.

Вывод по подразделу: корректная интерпретация результатов требует учитывать источник признака, степень уверенности, полноту динамического наблюдения и назначение каждого формата отчета.

### 3.10 Готовность проекта к демонстрации и дальнейшему сопровождению

Для защиты ВКР проект должен демонстрировать не только наличие исполняемой программы, но и завершенность инженерного цикла. Это означает, что должны быть представлены исходный код, структура сборки, набор тестов, примеры отчетов, документация по запуску и пояснение ограничений. В рассматриваемом проекте эти элементы связаны между

собой: тесты подтверждают заявленные блоки, а отчеты показывают, какие данные реально получает пользователь.

С точки зрения демонстрации важно показать четыре последовательных шага. Первый шаг - парсинг PE-файла и извлечение служебных структур. Второй шаг - статическая проверка этих структур и формирование признаков. Третий шаг - динамическое наблюдение на Windows x64, если оно допустимо для выбранного образца. Четвертый шаг - получение отчетов в форматах, удобных для человека, автоматической проверки и среды обратной разработки.

С точки зрения сопровождения важна воспроизводимая сборка. Использование CMake и проверка разными компиляторами снижают зависимость проекта от одной локальной конфигурации. Для статического режима это особенно важно, потому что он может применяться как переносимая утилита командной строки. Для динамического режима зависимость от Windows x64 является осознанным ограничением, связанным с Windows Debug API.

С точки зрения качества важна проверяемость отчетов. JSON-схема позволяет контролировать структуру машинного вывода, HTML-представление должно безопасно отображать данные, а CSV-форматы должны быть пригодны для последующей обработки. Если отчет нельзя проверить или воспроизвести, результат анализа теряет практическую ценность даже при корректной работе внутренних алгоритмов.

С точки зрения развития проект имеет понятное направление роста: расширение набора эвристик, более глубокое сопоставление unwind-данных с дизассемблированным прологом, поддержка дополнительных архитектур, улучшение визуального представления отчетов и интеграция с изолированными средами выполнения. Эти направления не требуют ломать базовую архитектуру, потому что основные блоки уже разделены.

Вывод по подразделу: проект можно демонстрировать как завершенную специализированную ВКР-разработку при условии, что в

защите явно раскрываются его назначение, границы применимости, четыре функциональных блока и результаты проверки.

### 3.11 Проверка полноты выполнения требований

Проверка полноты выполнения требований выполнялась путем сопоставления исходных требований с реализованными возможностями. Такой подход важен для ВКР, потому что позволяет показать не только результат работы программы, но и связь между целью, задачами, проектированием и проверкой. Если требование нельзя сопоставить с конкретной функцией или тестом, его выполнение остается недоказанным.

Требование к парсингу PE32+ AMD64 подтверждается тем, что инструмент читает заголовки, строит карту секций, обрабатывает каталог исключений и извлекает записи RUNTIME_FUNCTION. Проверка выполнялась как на синтетических образцах, так и на реальных обычных PE-файлах. Это показывает, что парсер работает не только на специально подготовленных минимальных файлах.

Требование к статическому анализу подтверждается набором эвристических признаков. В проверочном наборе аномальные образцы получают ненулевые признаки, а обычные проверенные файлы не дают массовых ложных срабатываний. Важным результатом является разбор большого числа runtime-функций без аварийного завершения и без выхода за границы буфера.

Требование к динамическому анализу подтверждается отдельными сценариями Windows x64. Проверка включает события исключений, регистрацию обработчиков и работу с динамическими таблицами функций. При этом контролируемые таймауты и невозможность запуска отдельных типов целей фиксируются как отдельные состояния, а не как неизвестные ошибки.

Требование к отчетности подтверждается наличием нескольких форматов вывода. JSON используется для машинной проверки, HTML - для просмотра человеком, CSV - для табличного анализа, Ghidra CSV - для интеграции со средой обратной разработки. Проверка схемы JSON и ручный

просмотр HTML позволяют убедиться, что отчет не является случайным текстовым выводом.

Сопоставление требований и результатов показывает, что заявленные задачи выполнены в рамках выбранной предметной области. Ограничения также описаны явно: поддерживается PE32+ AMD64, статический режим безопасен без запуска файла, а динамический режим работает только на Windows x64 и требует изоляции для неизвестных образцов.

### 3.12 Сценарий демонстрации проекта на защите

Для защиты целесообразно демонстрировать проект не как набор исходных файлов, а как последовательный рабочий сценарий. Сначала необходимо показать назначение инструмента: анализ EH/Unwind-метаданных Windows PE32+ AMD64. Затем следует пояснить, что проект состоит из четырех основных блоков: парсинг, статический анализ, динамический анализ и отчеты.

Первый демонстрационный шаг - запуск статического анализа на подготовленном PE-файле. На этом этапе важно показать, что файл не исполняется, а только читается как набор байтов. Комиссии следует объяснить, что инструмент строит карту секций, находит каталог исключений, извлекает RUNTIME_FUNCTION и декодирует UNWIND_INFO.

Второй шаг - просмотр найденных признаков. Здесь нужно показать не только итоговый балл, но и конкретные причины предупреждений. Например, можно указать на поврежденный диапазон функции, недостижимую структуру раскрутки или необычное размещение метаданных. Такой показ делает проект понятным даже для тех, кто не занимается обратной разработкой ежедневно.

Третий шаг - демонстрация динамического режима на безопасном контролируемом образце. Важно сразу проговорить, что динамика работает только на Windows x64 и выполняет файл под наблюдением отладчика. На защите не следует запускать неизвестные образцы без изоляции. Лучший

вариант - заранее подготовленный тестовый файл, который вызывает ожидаемые события.

Четвертый шаг - показ отчетов. JSON демонстрирует машинную структуру результата, HTML - удобный просмотр человеком, CSV - возможность табличной обработки, Ghidra CSV - связь с дальнейшей ручной обратной разработкой. Такой финал показывает, что проект не заканчивается консольной строкой, а дает полноценный результат для последующего анализа.

Правильный сценарий демонстрации помогает комиссии увидеть завершенность работы. Вместо сухого перечисления модулей показывается движение данных: входной PE-файл, статический разбор, признаки, динамическое подтверждение и итоговые отчеты.

### 3.13 Практическая значимость разработанного инструмента

Практическая значимость проекта связана с тем, что EH/Unwind-метаданные обычно остаются на границе внимания между PE-просмотрщиками, дизассемблерами и отладчиками. Эти структуры важны для восстановления функций, обработки исключений и понимания поведения программы, но их ручная проверка требует времени и внимательности. Инструмент автоматизирует именно этот узкий, но полезный слой анализа.

Для специалиста по информационной безопасности инструмент может использоваться как предварительный фильтр. Он не заменяет полноценную песочницу или систему детектирования вредоносного ПО, но помогает быстро выделить файлы с необычной структурой служебных данных. Это особенно полезно при большом количестве образцов, когда ручная проверка каждого файла невозможна.

Для специалиста по обратной разработке ценность состоит в ускорении первичной разметки. Автоматическое извлечение диапазонов функций, обработчиков и комментариев для Ghidra сокращает подготовительную работу. Аналитик получает не готовый ответ, а ориентиры, которые можно проверить в дизассемблере и сопоставить с машинным кодом.

Для учебного процесса проект полезен как пример связи между форматом исполняемого файла, ABI, безопасным парсингом, динамическим наблюдением и отчетностью. В отличие от абстрактной лабораторной работы, здесь есть законченный инструмент, тесты и проверяемые результаты. Это помогает объяснять низкоуровневые темы через практический продукт.

Для дальнейшего развития проект имеет понятные направления: расширение поддерживаемых архитектур, сопоставление unwind-данных с дизассемблированным прологом, более развитая визуализация отчетов, интеграция с изолированными средами запуска и расширение набора эвристик. При этом базовая архитектура уже допускает такие изменения без полной переработки.

Следовательно, практическая значимость работы состоит не в универсальной классификации файлов, а в создании специализированного инструмента, который делает узкий технический слой PE-анализа видимым, проверяемым и пригодным для дальнейшей ручной работы.

### 3.14 Подготовка материалов к эксплуатации

Для эксплуатации проекта недостаточно иметь только исполняемый файл. Пользователю необходимы инструкция по запуску, описание режимов, примеры отчетов, ограничения безопасности и сведения о проверке. Поэтому документация проекта должна объяснять не только параметры командной строки, но и смысл получаемого результата.

Минимальный эксплуатационный комплект включает собранный исполняемый файл, описание CLI, JSON-схему, пример HTML-отчета, пример CSV для Ghidra и сведения о тестовой проверке. Такой комплект позволяет повторить основные сценарии без изучения всего исходного кода. Для защиты ВКР это также удобно: комиссия видит не только программирование, но и готовность результата к демонстрации.

Отдельно необходимо описывать безопасный порядок работы с динамическим режимом. Неизвестные образцы не должны запускаться на

основной системе. Рекомендуемый порядок - статический анализ, просмотр отчета, принятие решения о необходимости динамики, запуск в изолированной Windows x64 среде, сохранение отчета и последующее сопоставление результатов.

При сопровождении важно сохранять совместимость отчетов. Если структура JSON меняется, должна обновляться схема и проверочные тесты. Если добавляются новые признаки, они должны получать описание, семейство и тестовый пример. Такой подход предотвращает ситуацию, когда проект постепенно превращается в набор неописанных проверок.

Для презентации проекта инвесторам или комиссии следует избегать чрезмерно внутренних деталей. Важно объяснять простую цепочку: файл анализируется без запуска, затем при необходимости наблюдается под отладчиком, после чего формируется отчет. Технические детали нужны для специалистов, но они должны быть привязаны к понятной задаче.

Подготовка материалов к эксплуатации завершает инженерный цикл проекта. Разработка считается законченной не тогда, когда код компилируется, а тогда, когда пользователь может безопасно запустить инструмент, понять отчет, воспроизвести проверку и увидеть ограничения результата.

### 3.15 Анализ достаточности тестового покрытия

Достаточность тестового покрытия в проекте оценивается не только количеством тестов, но и соответствием тестов рискам предметной области. Для анализатора PE-файлов ключевыми рисками являются выход за границы буфера, неправильное сопоставление RVA, ошибка чтения переменной структуры, неверное формирование признака и потеря события в динамическом режиме.

Модульные тесты закрывают локальные функции и граничные случаи. Они важны, потому что ошибка на низком уровне затем проявляется во всех последующих этапах. Если функция чтения числа или преобразования адреса

работает неверно, корректность эвристик уже не имеет значения. Поэтому базовый слой проверяется отдельно.

Синтетические образцы нужны для проверки известных аномалий. Их преимущество в том, что ожидаемый результат заранее понятен. Такой набор позволяет доказать, что инструмент действительно ловит заявленные случаи, а не просто успешно разбирает корректные файлы.

Обычные PE-файлы нужны для оценки ложных срабатываний. Если инструмент часто отмечает нормальные файлы как проблемные, его практическая ценность снижается. Поэтому проверка на реальном наборе обычных файлов является не менее важной, чем проверка на специально поврежденных образцах.

Динамические тесты проверяют не только наличие событий, но и устойчивость цикла наблюдения. Важными показателями являются валидность отчетов, отсутствие неожиданных отказов и отсутствие потерянных событий. Эти показатели демонстрируют, что динамический модуль способен работать не только в одном учебном сценарии.

Тестовое покрытие можно считать достаточным для целей ВКР, потому что оно охватывает основные риски проекта: парсинг, статическую оценку, отчеты и динамическое наблюдение. При промышленном развитии потребовалось бы расширять набор образцов и добавлять длительное fuzz-тестирование, но для заявленной темы текущий уровень проверки обоснован.

### 3.16 Оценка качества пользовательского результата

Качество пользовательского результата определяется тем, насколько быстро пользователь может понять вывод программы и перейти к проверке. Если отчет требует чтения исходного кода анализатора, результат нельзя считать удобным. Поэтому в проекте большое внимание уделяется структуре отчета и объяснению признаков.

Хороший отчет должен начинаться с общего состояния: удалось ли разобрать файл, сколько структур найдено, есть ли признаки и требуется ли ручная проверка. После этого пользователь должен видеть подробности:

какие функции, какие UNWIND_INFO, какие обработчики и какие причины предупреждений обнаружены.

Для специалиста важна адресная привязка. Предупреждение без адреса или индекса структуры сложно проверить. Поэтому признаки должны указывать на конкретный диапазон, запись, секцию или обработчик. Это позволяет открыть файл в дополнительном инструменте и подтвердить или опровергнуть вывод.

Для автоматизации важна стабильность JSON. Если поле меняет тип или исчезает без обновления схемы, внешние сценарии обработки ломаются. Поэтому JSON-формат должен рассматриваться как контракт. Его изменение требует обновления документации и тестов.

Для демонстрации важен HTML. Он позволяет показать результат комиссии без дополнительных программ. При этом HTML не должен быть перегружен необъясненными внутренними идентификаторами. В нем должны быть видны основные блоки: входной файл, статический анализ, динамика, признаки и рекомендации.

Качество результата можно оценить положительно, если пользователь понимает не только факт срабатывания, но и причину, место и дальнейшие действия. Именно такой подход заложен в текущую версию проекта.

### 3.17 Сравнение достигнутого результата с первоначальной целью

Первоначальная цель работы состояла в проектировании и разработке инструмента статико-динамического анализа EH/Unwind-метаданных Windows PE32+ AMD64. Достигнутый результат соответствует этой цели: реализованы парсинг PE, декодирование unwind-структур, эвристическая оценка, динамический сбор событий и формирование отчетов.

Задача анализа предметной области выполнена через изучение роли .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO и механизмов обработки исключений Windows x64. На основе этого анализа сформулированы требования и ограничения проекта. Это показывает связь между теоретической частью и реализацией.

Задача проектирования выполнена через модульную архитектуру. Каждый крупный блок имеет собственную ответственность, а данные передаются через внутреннюю модель результата. Такое решение соответствует сложности предметной области и позволяет развивать проект без полного переписывания.

Задача реализации выполнена в виде консольного инструмента. Выбор CLI оправдан: анализатор предназначен для технических специалистов, автоматизации и интеграции с другими средствами. Графический интерфейс мог бы быть направлением развития, но не является обязательным для доказательства основной идеи.

Задача проверки выполнена через несколько уровней тестирования и проверочные наборы. Полученные результаты показывают, что инструмент корректно обрабатывает заявленные сценарии, формирует отчеты и не демонстрирует неожиданных отказов в проверенных динамических запусках.

Сравнение цели и результата показывает, что проект завершен в рамках темы ВКР. Оставшиеся направления развития относятся к расширению возможностей, а не к устранению недоделок базовой архитектуры.

### 3.18 Рекомендации по представлению работы на нормоконтроле

Для нормоконтроля важно, чтобы пояснительная записка выглядела как единый документ, а не как набор технических приложений. Поэтому основной текст должен содержать постановку задачи, анализ предметной области, проектирование, реализацию, проверку и выводы. Приложения должны дополнять основной текст, но не заменять его.

В титульной части необходимо проверить локальные данные: ФИО студента, руководителя, нормоконтролера, группу, кафедру, направление подготовки и год. Эти сведения нельзя оставлять условными, если документ передается в финальную проверку. Технически документ подготовлен, но персональные поля должны быть заполнены владельцем работы.

В содержании должны совпадать названия разделов и страницы. Поэтому после любых ручных правок в Word необходимо обновлять поля

содержания. Если содержание не обновить, даже качественный текст может получить замечание нормоконтроля из-за расхождения страниц.

Список источников должен оставаться не случайным набором ссылок, а опорой для текста. В работе используются источники по PE-формату, обработке исключений Windows, Windows Debug API, стандартам оформления, инструментам сборки и обратной разработке. При добавлении новых утверждений следует добавлять и источники, если утверждения выходят за рамки собственного проекта.

Приложения должны иметь смысловую связь с текстом. В текущей структуре приложения раскрывают техническое задание, параметры командной строки, результаты проверки, карту признаков, алгоритмы, JSON-структуру и эксплуатационные материалы. Это лучше, чем большие листинги без пояснения, потому что приложения помогают защищать работу.

Перед сдачей на нормоконтроль рекомендуется открыть PDF и просмотреть титульный лист, задание, содержание, начало каждого раздела, таблицы, рисунки, список источников и приложения. Такая ручная проверка занимает немного времени, но позволяет поймать ошибки, которые автоматические средства часто пропускают.

### 3.19 Оценка рисков внедрения

Внедрение разработанного инструмента связано не только с техническими, но и с организационными рисками. Основной риск состоит в неправильном понимании результата пользователем. Если воспринимать эвристический признак как окончательное доказательство вредоносности, инструмент будет применяться некорректно. Поэтому в документации и в демонстрации необходимо подчеркивать, что результат является основанием для ручной проверки.

Второй риск связан с безопасностью динамического режима. Статический анализ не исполняет файл, но динамический режим запускает его под наблюдением отладчика. Для неизвестных образцов это допустимо только в изолированной среде. При внедрении необходимо закрепить

правило: сначала выполняется статический анализ, затем принимается решение о необходимости динамики, и только после этого используется подготовленная Windows x64 среда.

Третий риск связан с изменением форматов отчетов. Если проект используется в автоматизации, внешние сценарии могут зависеть от JSON-полей. Поэтому любые изменения схемы должны сопровождаться обновлением документации и тестов. Иначе инструмент будет формально работать, но нарушит совместимость с уже настроенными процессами.

Четвертый риск относится к расширению эвристик. Новые признаки могут повысить чувствительность, но одновременно увеличить число ложных срабатываний. Поэтому каждое новое правило должно сопровождаться тестовым примером, описанием семейства, объяснением причины и проверкой на обычных PE-файлах.

Пятый риск связан с человеческим фактором. Пользователь может не знать, что отсутствие динамического события не доказывает отсутствие механизма в программе. Для снижения этого риска отчет должен явно показывать полноту наблюдения, таймауты, ограничения запуска и состояние завершения процесса.

Оценка рисков показывает, что проект готов к демонстрационному и лабораторному применению, но требует аккуратного регламента при работе с неизвестными файлами. Это нормальное ограничение для инструмента анализа исполняемых файлов и оно не противоречит цели ВКР.

### 3.20 Итоговая оценка готовности программного средства

Итоговая готовность программного средства оценивается по нескольким признакам: наличие реализованного функционала, воспроизводимая сборка, проверочные наборы, документация, понятные ограничения и пригодность отчетов для пользователя. В разработанном проекте все перечисленные признаки присутствуют.

Функциональная готовность подтверждается тем, что реализованы четыре ключевых блока: парсинг, статический анализ, динамический анализ

и отчетность. Каждый блок имеет самостоятельное назначение и связан с общей целью работы. Это позволяет показывать проект не как набор разрозненных функций, а как законченный конвейер анализа.

Техническая готовность подтверждается структурой исходного кода и сборочной системой. Проект можно собирать разными компиляторами, проверять тестами и запускать в режимах, соответствующих поставленным задачам. При этом динамический модуль не скрывает свою зависимость от Windows x64.

Проверочная готовность подтверждается результатами тестирования. Выполнены модульные проверки, синтетические сценарии, анализ обычных PE-файлов и динамические проверки. Полученные результаты показывают, что инструмент работает в заявленных границах и формирует валидные отчеты.

Документационная готовность подтверждается наличием пояснительной записки, описания архитектуры, требований, ограничений, алгоритмов, отчетных форматов и приложений. Это важно для защиты, потому что комиссия оценивает не только код, но и способность автора объяснить принятые инженерные решения.

Следовательно, программное средство можно считать готовым к представлению в рамках ВКР. Оно имеет завершенную функциональность, проверяемую архитектуру, понятную область применения и направления дальнейшего развития, не маскирующие уже выполненный объем работы.

## ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы спроектирован и разработан инструмент статико-динамического анализа EH/Unwind-метаданных Windows PE32+ AMD64. Работа была направлена на решение практической задачи: автоматизировать извлечение и проверку служебных структур обработки исключений, которые в обычных инструментах часто требуют ручной интерпретации.

В аналитической части рассмотрена предметная область, связанная с форматом PE32+ AMD64, таблицей .pdata, структурами UNWIND_INFO и механизмами обработки исключений Windows x64. Показано, что EH/Unwind-слой является важным источником сведений о функциях и обработчиках, а его повреждение или нестандартное размещение может затруднить отладку и обратную разработку.

Проведен анализ существующих средств: PE-просмотрщиков, дизассемблеров, отладчиков и песочниц. Установлено, что эти инструменты полезны, но не закрывают узкую задачу объяснимого отчета по EH/Unwind-метаданным. На основании анализа сформированы требования к инструменту: безопасный статический разбор, эвристическая оценка, динамический сбор Windows x64 и несколько форматов отчетности.

В проектной части разработана модульная конвейерная архитектура. В ней выделены блоки парсинга PE, декодирования unwind-структур, статической оценки, динамического наблюдения и генерации отчетов. Такое разделение позволило сохранить ясные границы ответственности модулей и сделать результат пригодным для модульного и интеграционного тестирования.

В ходе реализации создан парсер PE32+ AMD64, декодер RUNTIME_FUNCTION и UNWIND_INFO, эвристическая модель признаков, динамический сборщик событий через Windows Debug API, генераторы отчетов JSON, HTML, CSV и Ghidra CSV. Для JSON-отчета задана схема, а для Ghidra подготовлен импорт, позволяющий переносить функции и комментарии в среду обратной разработки.

Проверка работы выполнена на нескольких уровнях. Модульные тесты подтвердили корректность отдельных функций и граничных случаев. Синтетический набор позволил проверить известные аномалии. Набор обычных PE-файлов подтвердил отсутствие массовых ложных срабатываний на проверенных образцах. Динамические сценарии подтвердили работу наблюдения за VEH, VCH, UEF и runtime function tables.

Результаты проверки показывают, что поставленная цель достигнута. Инструмент формирует воспроизводимый отчет, не смешивает подозрительность с доказательством вредоносности, показывает причины выводов и сохраняет ограничения динамического режима. Он может использоваться в учебном процессе и в лабораторном сценарии первичного анализа PE-файлов.

Практическая значимость работы состоит в том, что разработанное средство сокращает объем ручной проверки EH/Unwind-метаданных и подготавливает данные для дальнейшего анализа. Особенно важна объяснимость: каждый признак связан с конкретной структурой, адресом и причиной, поэтому специалист может подтвердить или отклонить вывод инструмента.

Дальнейшее развитие проекта может включать поддержку новых архитектур, сопоставление unwind-структур с дизассемблированным прологом, расширение статико-динамической корреляции, графическое представление отчетов и интеграцию с песочницами. При этом базовый принцип проекта должен сохраняться: результат анализа должен быть проверяемым, а границы применимости - явно описанными.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ


1. Microsoft. PE Format : сайт. - URL: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format (дата обращения: 20.06.2026).

2. Microsoft. x64 exception handling : сайт. - URL: https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64 (дата обращения: 20.06.2026).

3. Microsoft. RUNTIME_FUNCTION structure : сайт. - URL: https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-runtime_function (дата обращения: 20.06.2026).

4. Microsoft. UNWIND_INFO structure : сайт. - URL: https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64 (дата обращения: 20.06.2026).

5. Microsoft. Debugging Functions : сайт. - URL: https://learn.microsoft.com/en-us/windows/win32/debug/debugging-functions (дата обращения: 20.06.2026).

6. Microsoft. AddVectoredExceptionHandler function : сайт. - URL: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-addvectoredexceptionhandler (дата обращения: 20.06.2026).

7. Microsoft. AddVectoredContinueHandler function : сайт. - URL: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-addvectoredcontinuehandler (дата обращения: 20.06.2026).

8. Microsoft. SetUnhandledExceptionFilter function : сайт. - URL: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-setunhandledexceptionfilter (дата обращения: 20.06.2026).

9. Microsoft. RtlAddFunctionTable function : сайт. - URL: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtladdfunctiontable (дата обращения: 20.06.2026).

10. Microsoft. RtlInstallFunctionTableCallback function : сайт. - URL: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtlinstallfunctiontablecallback (дата обращения: 20.06.2026).

11. Intel 64 and IA-32 Architectures Software Developer's Manual : сайт. - URL: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html (дата обращения: 20.06.2026).

12. ISO/IEC 9899:2018. Programming languages - C. - Geneva : International Organization for Standardization, 2018.

13. CMake Documentation : сайт. - URL: https://cmake.org/cmake/help/latest/ (дата обращения: 20.06.2026).

14. Ninja, a small build system with a focus on speed : сайт. - URL: https://ninja-build.org/manual.html (дата обращения: 20.06.2026).

15. JSON Schema : сайт. - URL: https://json-schema.org/ (дата обращения: 20.06.2026).

16. Ghidra Software Reverse Engineering Framework : сайт. - URL: https://ghidra-sre.org/ (дата обращения: 20.06.2026).

17. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. - Введ. 2018-07-01. - Москва : Стандартинформ, 2017. - 24 с. - URL: https://docs.cntd.ru/document/1200157208 (дата обращения: 20.06.2026).

18. ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления : национальный стандарт Российской Федерации. - Введ. 2019-07-01. - Москва : Стандартинформ, 2018. - URL: https://docs.cntd.ru/document/1200161674 (дата обращения: 20.06.2026).

19. Алферьева Т. И. Методические рекомендации для подготовки и оформления выпускных квалификационных работ : методические рекомендации / Т. И. Алферьева, В. Н. Васина, Д. Б. Шадрин. - Екатеринбург : УрФУ, 2019. - 78 с.

20. Sikorski M. Practical Malware Analysis : The Hands-On Guide to Dissecting Malicious Software / M. Sikorski, A. Honig. - San Francisco : No Starch Press, 2012. - 800 p.

21. Dang G. Practical Reverse Engineering : x86, x64, ARM, Windows Kernel, Reversing Tools, and Obfuscation / G. Dang, A. Gazet, E. Bachaalany. - Indianapolis : Wiley, 2014. - 384 p.

22. Eilam E. Reversing: Secrets of Reverse Engineering / E. Eilam. - Indianapolis : Wiley, 2005. - 624 p.

23. National Institute of Standards and Technology. Guide to Malware Incident Prevention and Handling for Desktops and Laptops : SP 800-83 Rev. 1. - Gaithersburg : NIST, 2013. - URL: https://csrc.nist.gov/publications/detail/sp/800-83/rev-1/final (дата обращения: 20.06.2026).

24. Microsoft. Visual Studio C/C++ documentation : сайт. - URL: https://learn.microsoft.com/en-us/cpp/ (дата обращения: 20.06.2026).

25. MinGW-w64 project : сайт. - URL: https://www.mingw-w64.org/ (дата обращения: 20.06.2026).

26. LLVM. libFuzzer - a library for coverage-guided fuzz testing : сайт. - URL: https://llvm.org/docs/LibFuzzer.html (дата обращения: 20.06.2026).

27. OWASP. Cross Site Scripting Prevention Cheat Sheet : сайт. - URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html (дата обращения: 20.06.2026).

28. GitHub Docs. About continuous integration : сайт. - URL: https://docs.github.com/en/actions/concepts/building-and-testing/about-continuous-integration (дата обращения: 20.06.2026).

29. Microsoft. Windows Debugger documentation : сайт. - URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/ (дата обращения: 20.06.2026).

30. Проект EH/Unwind Analyzer. Архитектура, исходный код, тестовые наборы и отчеты проверки : локальные материалы выпускной квалификационной работы. - Екатеринбург, 2026.

# ПРИЛОЖЕНИЕ А

(обязательное)

## Частное техническое задание

Таблица А.1 - Основные положения технического задания

| Параметр | Описание |
|---|---|
| Назначение | Автоматический анализ EH/Unwind-метаданных PE32+ AMD64 |
| Входные данные | путь к PE-файлу и параметры командной строки |
| Основной режим | статический анализ без запуска файла |
| Дополнительный режим | динамический сбор событий Windows x64 |
| Выходные данные | консольный вывод, JSON, HTML, CSV, Ghidra CSV |
| Ограничения | PE32+ AMD64; динамика только Windows x64 |

## ПРИЛОЖЕНИЕ Б

(обязательное)

### Основные параметры командной строки

Таблица Б.1 - CLI-параметры инструмента

| Опция | Назначение |
|---|---|
| --dump | вывод записей таблицы RUNTIME_FUNCTION |
| --ops | вывод декодированных операций UNWIND_INFO |
| --heuristics | запуск эвристической оценки |
| --strict-abi | добавление формальных ABI-диагностик |
| --dynamic | запуск динамического режима Windows x64 |
| --timeout-ms N | таймаут динамического режима |
| --json path | запись полного JSON-отчета |
| --html path | запись HTML-отчета |
| --csv path | запись таблицы функций |
| --ghidra-csv path | экспорт для Ghidra |

## ПРИЛОЖЕНИЕ В

(обязательное)

## Сводка результатов проверки

Таблица В.1 - Проверочная сводка проекта

| Показатель | Результат |
|---|---|
| MinGW-w64 Release CTest | 16 из 16 |
| MSVC Release CTest | 16 из 16 |
| Синтетический статический набор | 14 из 14 валидных отчетов |
| Обычный реальный PE-набор | 106 из 106 валидных отчетов |
| Разобранные runtime-функции | 174 795 |
| Динамические smoke-сценарии | 4 из 4 |
| Полный динамический набор | 124 из 124 валидных отчетов |
| Неожиданные динамические отказы | 0 |
| Потерянные события | 0 |

## ПРИЛОЖЕНИЕ Г

(справочное)

## Ключевые семейства эвристических признаков

Таблица Г.1 - Семейства признаков

| Семейство | Смысл |
|---|---|
| table_integrity | повреждения и противоречия в таблице RUNTIME_FUNCTION |
| unwind_integrity | ошибки структуры UNWIND_INFO и связанных tail-данных |
| metadata_placement | размещение метаданных в необычных, исполняемых или writable-секциях |
| unwind_reuse | массовое повторное использование одного блока данных раскрутки |
| handler_dominance | доминирование одного обработчика над большим числом функций |

## ПРИЛОЖЕНИЕ Д
(обязательное)

### Расширенное техническое задание

Расширенное техническое задание фиксирует не только назначение программы, но и границы ее применения. Инструмент предназначен для анализа PE32+ AMD64 файлов с акцентом на EH/Unwind-метаданные Windows. Основной режим работы является статическим, то есть входной файл не исполняется. Динамический режим используется отдельно и требует Windows x64.

Основной пользователь инструмента - специалист, которому нужно быстро получить техническую карту служебных структур файла. Пользователь должен видеть, какие функции обнаружены, какие данные раскрутки связаны с ними, какие признаки сформированы и какие участки требуют ручной проверки.

Таблица Д.1 - расширенное техническое задание

| Блок | Требование | Критерий приемки |
|---|---|---|
| Парсинг | Чтение PE32+ AMD64, секций и каталога исключений | Файл разбирается без выхода за границы буфера |
| Статика | Декодирование RUNTIME_FUNCTION и UNWIND_INFO | Формируется список функций и диагностик |
| Эвристики | Выявление структурных аномалий | Каждый признак имеет причину и адресную привязку |
| Динамика | Наблюдение Windows x64 через отладочный цикл | События фиксируются с учетом таймаута и полноты |
| Отчеты | JSON, HTML, CSV, Ghidra CSV | Результат воспроизводим и пригоден для ручной проверки |

## ПРИЛОЖЕНИЕ Е
(обязательное)

### Алгоритмы обработки данных

Алгоритмы проекта построены как конвейер. Каждый этап принимает результат предыдущего этапа и возвращает либо нормализованные данные, либо диагностируемое состояние. Такой подход упрощает отладку и позволяет показывать на защите, где именно возникает ошибка или признак.

В статическом режиме конвейер не переходит к следующей стадии, пока не проверена корректность входа текущей стадии. В динамическом режиме каждое событие получает статус обработки, а итоговый отчет содержит информацию о полноте наблюдения.

Таблица Е.1 - алгоритмы обработки данных

| N | Этап | Основное действие |
|---|---|---|
| 1 | Чтение файла | Загрузка байтов и проверка минимального размера |
| 2 | PE-парсинг | Проверка заголовков, секций и каталога исключений |
| 3 | RVA mapping | Преобразование адресов только через карту секций |
| 4 | Unwind decode | Пошаговое чтение UNWIND_INFO и дополнительных полей |
| 5 | Heuristics | Формирование признаков и уровня уверенности |
| 6 | Report | Сериализация единой модели результата |

## ПРИЛОЖЕНИЕ Ж
(обязательное)

### Структура JSON-отчета

JSON-отчет является основой автоматической проверки результата. Он хранит не только итоговую оценку, но и исходные данные анализа: структуру PE, функции, unwind-сведения, признаки, динамические события и диагностические состояния. Благодаря этому отчет можно использовать в регрессионных тестах.

При изменении структуры JSON необходимо обновлять схему и тестовые ожидания. Это требование особенно важно при развитии проекта, потому что отчет является внешним контрактом между анализатором и пользователем.

Таблица Ж.1 - структура json-отчета

| Раздел | Содержимое | Назначение |
|---|---|---|
| input | путь, размер, контрольные сведения | идентификация анализируемого файла |
| pe | заголовки, секции, каталоги | описание структуры образа |
| runtime_functions | диапазоны функций и ссылки на unwind | карта функций AMD64 |
| unwind | операции раскрутки, флаги, обработчики | модель EH/Unwind-данных |
| findings | семейство, серьезность, адрес, пояснение | список признаков |
| dynamic | события, таймауты, полнота наблюдения | результаты Windows x64 режима |
| summary | балл, уверенность, рекомендация | итог для пользователя |

## ПРИЛОЖЕНИЕ И
(справочное)

### Матрица тестовых проверок

Матрица тестовых проверок показывает, какие части проекта подтверждаются отдельными наборами тестов. Она нужна не для замены текстового описания тестирования, а для компактного сопоставления требований и проверочных сценариев.

Наборы проверок разделены по уровню: модульные тесты, синтетические образцы, обычные PE-файлы, динамические smoke-сценарии и полный динамический набор. Такое разделение позволяет отдельно проверять корректность функций, устойчивость парсера и поведение динамического режима.

Таблица И.1 - матрица тестовых проверок

| Уровень | Что проверяется | Ожидаемый результат |
|---|---|---|
| Модульные тесты | границы чтения, строки, парсер, отчеты | все тесты завершаются успешно |
| Синтетика | заранее известные аномалии | аномальные образцы получают признаки |
| Обычные PE | ложные срабатывания | нет массовых ненулевых оценок |
| Динамический smoke | VEH/VCH/UEF/runtime tables | фиксируются ожидаемые события |
| Полный динамический набор | устойчивость запуска и отчетов | неожиданные отказы отсутствуют |

## ПРИЛОЖЕНИЕ К
(справочное)

### Карта эвристических признаков

Карта эвристических признаков помогает объяснить, почему инструмент формирует предупреждение. Каждый признак должен относиться к понятному семейству и иметь техническую причину. Это делает результат пригодным для ручной проверки и снижает риск произвольной интерпретации.

В дальнейшем карта признаков может расширяться. При добавлении нового признака необходимо указывать семейство, проверяемое условие, уровень серьезности, возможные причины ложного срабатывания и тестовый пример.

Таблица К.1 - карта эвристических признаков

| Семейство | Проверяемое условие | Что должен увидеть пользователь |
|---|---|---|
| table_integrity | повреждение или противоречие таблицы функций | адрес записи и причина нарушения |
| unwind_integrity | ошибка структуры UNWIND_INFO | поле структуры и характер ошибки |
| metadata_placement | необычное размещение служебных данных | секция, права доступа и RVA |
| unwind_reuse | массовое повторное использование данных | связанные функции и общий блок |
| handler_dominance | доминирование одного обработчика | адрес обработчика и число ссылок |

## ПРИЛОЖЕНИЕ Л
(справочное)

### Чек-лист подготовки к нормоконтролю

Чек-лист предназначен для финальной ручной проверки документа перед передачей на нормоконтроль. Он не заменяет методические указания, но помогает быстро пройти по типовым местам, где чаще всего возникают замечания.

Особое внимание следует уделить титульному листу, заданию, содержанию, нумерации страниц, названиям таблиц и рисунков, списку источников и приложениям. Эти элементы проверяются визуально и должны совпадать с требованиями кафедры.

Таблица Л.1 - чек-лист подготовки к нормоконтролю

| Проверяемый элемент | Что проверить | Статус |
|---|---|---|
| Титульный лист | ФИО, тема, город, год, подписи | проверить вручную |
| Задание | тема, исходные данные, календарный план | проверить вручную |
| Содержание | страницы после обновления полей Word | обновить перед сдачей |
| Основной текст | 45-70 страниц без приложений | проверить по PDF |
| Источники | не менее 25, оформление по ГОСТ | проверить список |
| Приложения | наличие ссылочной и технической ценности | проверить состав |

## ПРИЛОЖЕНИЕ М
(справочное)

### Порядок безопасного запуска

Порядок запуска нужен для того, чтобы пользователь не смешивал статический и динамический режимы. Первичный анализ должен выполняться статически. Динамический режим используется только тогда, когда есть безопасная среда Windows x64 и понятный сценарий проверки.

При работе с неизвестными файлами необходимо использовать изолированную виртуальную машину. Результаты анализа следует сохранять в отдельную папку, чтобы не смешивать отчеты разных запусков и не потерять JSON, HTML и CSV материалы.

Таблица М.1 - порядок безопасного запуска

| Шаг | Действие | Комментарий |
|---|---|---|
| 1 | Подготовить файл и папку отчетов | не запускать образец вручную |
| 2 | Выполнить статический анализ | получить первичный JSON/HTML |
| 3 | Просмотреть признаки | определить необходимость динамики |
| 4 | Подготовить изолированную Windows x64 | снимок виртуальной машины желателен |
| 5 | Запустить динамический режим | задать таймаут и сохранить отчет |
| 6 | Сопоставить результаты | сравнить статические признаки и события |

## ПРИЛОЖЕНИЕ Н
(справочное)

### Матрица связи разделов, задач и результатов

Матрица связи помогает показать, что структура пояснительной записки не случайна. Каждый раздел связан с одной или несколькими задачами ВКР, а каждая задача приводит к конкретному результату: требованию, модулю, проверке или отчетному материалу.

Такая матрица удобна при подготовке к защите. По ней можно быстро восстановить логику ответа: зачем выбран проект, что изучалось, что спроектировано, что реализовано и чем подтверждена работоспособность.

Таблица Н.1 - матрица связи разделов, задач и результатов

| Раздел | Связанная задача | Результат |
|---|---|---|
| Введение | обосновать актуальность и цель | цель, объект, предмет, задачи |
| Раздел 1 | изучить предметную область | требования и ограничения |
| Раздел 2 | спроектировать и реализовать инструмент | архитектура и алгоритмы |
| Раздел 3 | проверить работоспособность | результаты тестирования |
| Заключение | сопоставить цель и результат | итоговые выводы |
| Приложения | дать технические материалы | задание, алгоритмы, чек-листы |

## ПРИЛОЖЕНИЕ П
(справочное)

### Краткое руководство пользователя

Руководство пользователя описывает минимальный порядок работы с инструментом. Оно предназначено для человека, который не изучал исходный код, но должен запустить анализ, получить отчет и понять дальнейшие действия.

Перед запуском необходимо определить, какой режим требуется. Для первичного исследования используется статический анализ. Динамический режим применяется только на Windows x64 и только в изолированной среде, если файл неизвестен или потенциально опасен.

После завершения анализа пользователь просматривает общий статус, список признаков и ссылки на конкретные структуры. Если признаки отсутствуют, это означает только отсутствие выявленных признаков в рамках реализованной модели. Если признаки есть, файл передается на ручную проверку.

Таблица П.1 - краткое руководство пользователя

| Ситуация | Рекомендуемое действие |
|---|---|
| Нужно быстро проверить файл | запустить статический анализ и открыть HTML-отчет |
| Нужно автоматизировать проверку | использовать JSON и схему отчета |
| Нужно изучить функции в Ghidra | сформировать Ghidra CSV и импортировать данные |
| Нужно подтвердить runtime-события | запустить динамический режим в изоляции Windows x64 |
| Есть таймаут динамики | считать результат неполным, а не отрицательным |

## ПРИЛОЖЕНИЕ Р
(справочное)

### Форма протокола проверки

Форма протокола проверки может использоваться при демонстрации проекта или при повторной проверке после изменений. Она помогает фиксировать, какие режимы запускались, какие отчеты получены и какие ограничения были замечены.

Такой протокол полезен для сопровождения: если в будущем изменятся эвристики или формат отчета, можно сравнить новый результат с предыдущим и увидеть, какие изменения являются ожидаемыми, а какие требуют анализа.

Протокол не заменяет автоматические тесты, но дополняет их ручной проверкой ключевых пользовательских сценариев. В дипломной работе это помогает показать, что результат проверяется не только разработчиком в коде, но и как готовое средство.

Таблица Р.1 - форма протокола проверки

| Поле | Заполняемое значение |
|---|---|
| Дата проверки | указать дату ручной проверки |
| Версия сборки | указать сборку или конфигурацию |
| Входной файл | путь или краткое описание образца |
| Статический режим | успешно / ошибка / ограничение |
| Динамический режим | не запускался / успешно / таймаут / ограничение |
| Отчеты | JSON / HTML / CSV / Ghidra CSV |
| Вывод проверяющего | краткое заключение по результату |

# ПРИЛОЖЕНИЕ С

(обязательное)

## Актуализированная матрица требований, признаков, тестов и отчетного контракта

Данное приложение добавлено к финальной редакции пояснительной записки для синхронизации текста ВКР с текущей реализацией EH/Unwind Analyzer 1.0.0. Его назначение - явно показать требования к итоговым материалам, структуру 27 диагностических признаков, связь 16 CTest-сценариев с 78 PE-бинарниками, текущую версию JSON Schema 1.2.0 и границы корректной защиты проекта.

## С.1 Требования к итоговой пояснительной записке и презентации

Итоговый комплект должен быть построен по академической логике: проблема предметной области, актуальность, цель и задачи, требования, анализ аналогов, архитектура, реализация, проверка, результаты, ограничения и развитие. Такая структура повторяет успешную защитную модель, но содержание адаптировано к низкоуровневому инструменту анализа PE32+ AMD64 EH/Unwind-метаданных.

Таблица С.1 - Требования к итоговому защитному комплекту

| Код | Требование | Как раскрывается |
|---|---|---|
| R1 | Не называть проект универсальным антивирусом | Проект выявляет технические аномалии EH/Unwind-слоя, а не выносит итоговый malware-verdict. |
| R2 | Показать, что проект является системой анализа | Не только parser: есть static map, findings, score, confidence, dynamic evidence, reports и Ghidra workflow. |
| R3 | Разделить static и dynamic | Static отвечает, какую карту обещает файл; dynamic отвечает, какие exception-related события реально наблюдались. |
| R4 | Объяснить 27 findings | Каждый признак должен иметь code, family, смысл, severity/score и проверяемое evidence. |
| R5 | Объяснить тестирование без путаницы чисел | 16 CTest-сценариев, 27 findings и 78 PE-бинарников относятся к разным уровням проверки. |
| R6 | Зафиксировать ограничения | PE32+ AMD64, dynamic только Windows x64, untrusted samples только в VM, нет автоматического сравнения UWOP с прологом .text. |

## С.2 Актуальный паспорт реализации

Таблица С.2 - Паспорт текущей реализации

| Параметр | Значение | Пояснение |
|---|---|---|
| Версия продукта | 1.0.0 | Фиксируется в CMakeLists.txt и include/app_info.h. |
| Версия JSON Schema | 1.2.0 | Текущая схема находится в docs/report.schema.json и проверяется тестами. |
| Производственное C-ядро | 7 771 строка | src/*.c и include/*.h: parser, decoder, heuristics, dynamic collector, reports, CLI. |
| Выбранная кодовая база | около 40,5 тыс. строк | С учетом тестов, Python-валидаторов, release scripts, Ghidra importer и batch-файлов. |
| Тестовые PE-бинарники | 78 | Расширенный демонстрационный корпус: static, dynamic и mixed образцы. |
| Статический benign corpus | 106/106 valid, score > 0: 0 | Проверяет дисциплину ложных срабатываний на обычных PE-файлах. |
| Full dynamic corpus | 124/124 valid reports | Проверяет Windows dynamic collector и классификацию outcomes. |
| Windows artifact SHA-256 | b1cee8678cdf21d96e2921625ffabc3d5c1be4d41cb022d4d0954ef3692df63a | Фиксирует проверенный Windows x64 артефакт релиза. |

## С.3 Полная матрица 27 диагностических признаков

В текущей версии проекта признак означает не ярлык “вредоносно”, а проверяемое техническое наблюдение. Признаки покрывают пять семейств: целостность таблицы функций, целостность UNWIND_INFO, размещение metadata, reuse unwind-структур и dominance обработчиков. Комбинированные признаки усиливают вывод только тогда, когда несколько слоев evidence согласуются между собой.

Таблица C.3 - 27 findings EH/Unwind Analyzer

| N | Code | Family | Смысл проверки |
|---|---|---|---|
| 1 | pe_section_raw_oob | table_integrity | секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным |
| 2 | pe_section_virtual_oob | table_integrity | виртуальный диапазон секции выходит за объявленный SizeOfImage |
| 3 | pe_section_raw_overlap | table_integrity | две секции пересекаются в файле и могут трактовать одни байты по-разному |
| 4 | pe_section_virtual_overlap | table_integrity | две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA |
| 5 | pe_section_misaligned | table_integrity | адреса или размеры секций не согласованы с file/section alignment |
| 6 | pe_exception_directory_oob | table_integrity | Exception Directory находится вне допустимого образа или не переводится в валидные байты |
| 7 | no_runtime_functions | table_integrity | для AMD64 образа не найдено записей RUNTIME_FUNCTION |
| 8 | invalid_ranges | table_integrity | в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон |
| 9 | rf_oob | table_integrity | диапазон функции выходит за границы валидного image/section context |
| 10 | unwind_rva_oob | table_integrity | адрес UNWIND_INFO не отображается в валидный file offset или секцию |
| 11 | rf_unsorted | table_integrity | таблица функций не отсортирована по BeginAddress |
| 12 | rf_overlaps | table_integrity | диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции |
| 13 | duplicate_begin | table_integrity | несколько записей имеют одинаковое начало функции |
| 14 | unwind_parse_fail | unwind_integrity | UNWIND_INFO невозможно разобрать до конца по формату Windows x64 |
| 15 | unwind_anomalies | unwind_integrity | UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern |
| 16 | unwind_reuse_moderate | unwind_reuse | заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов |
| 17 | unwind_reuse_strong | unwind_reuse | слишком большая доля функций использует один и тот же UnwindInfoRVA |
| 18 | handler_dominance | handler_dominance | один handler target обслуживает подозрительно много функций |
| 19 | unwind_in_exec_section | metadata_placement | UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata |
| 20 | unwind_unusual_section | metadata_placement | UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге |
| 21 | unwind_writable_section | metadata_placement | UNWIND_INFO лежит в секции с правом записи |
| 22 | handler_nonexec_section | metadata_placement | handler RVA указывает в неисполняемую или неизвестную секцию |
| 23 | handler_writable_section | metadata_placement | handler RVA указывает в writable секцию |
| 24 | placement_exec_nonexec_handler_combo | metadata_placement | одновременно странно размещены unwind metadata и handler target |
| 25 | placement_integrity_combo | unwind_integrity | placement anomaly подтверждается повреждением или странностью UNWIND_INFO |
| 26 | reuse_dominance_combo | unwind_reuse | массовое повторное использование unwind metadata сочетается с доминированием handler |
| 27 | writable_metadata_handler_combo | metadata_placement | и unwind metadata, и handler target связаны с writable context |

## C.4 Как объяснять связь 16 тестов, 27 признаков и 78 бинарников

На защите нельзя говорить так, будто 16 тестов должны один к одному соответствовать 27 признакам. Это разные сущности. CTest-сценарий - это автоматизированный сценарий проверки кода. Finding - это диагностическое правило. PE-бинарник - это входной артефакт, на котором может сработать один или несколько findings.

Таблица C.4 - Разные уровни тестовой доказательности

| Число/набор | Что означает | Как объяснять комиссии |
|---|---|---|
| 16 CTest-сценариев | Проверяют кодовую базу: unit, regression, schema, semantic, CLI, release metadata, hygiene, mutation fuzz. | Это сценарии качества реализации. |
| 27 findings | Проверяют диагностическую модель: какие технические сигналы проект умеет формировать. | Это признаки анализа. |
| 78 PE-бинарников | Дают контролируемые входы: baseline, malformed static, dynamic, mixed. | Это демонстрационный и регрессионный корпус. |

| Число/набор | Что означает | Как объяснять комиссии |
|---|---|---|
| 106 real benign PE | Проверяют, что обычные файлы не получают ненулевой score без причины. | Это контроль false positive discipline. |
| 124 dynamic reports | Проверяют dynamic collector, валидность отчетов и корректную классификацию runtime outcomes. | Это Windows-only evidence. |

## C.5 Отчетный контракт и данные, необходимые реверс-инженеру

Отчет проекта должен быть доказательным, а не декларативным. Минимально полезный finding показывает код признака, семейство, серьезность, confidence, affected count, representative RVA/section и ручные шаги проверки. JSON служит машинным контрактом для будущей БД/backend, HTML - человекочитаемым dashboard, CSV - табличной выгрузкой, Ghidra CSV - мостом в интерактивный reverse engineering.

Таблица C.5 - Ключевые блоки отчета

| Блок | Что содержит | Зачем нужен |
|---|---|---|
| input/report_metadata | Путь, SHA-256, размер, версия инструмента, версия schema, параметры запуска. | Позволяет воспроизвести анализ. |
| pe_validation/sections | Секции, raw/virtual ranges, permissions, alignment, Exception Directory. | Показывает карту адресов и доверие к mapping. |
| functions | Begin/End, size, section, UnwindInfoRVA, parse status, unwind summary. | Позволяет читать .pdata/.xdata как таблицу функций. |
| heuristic_analysis/findings | 27 named findings, family, severity, score, evidence. | Показывает, почему возникла тревога. |
| assessment | Score, evidence level, confidence, case strength, manual review flag. | Дает triage-вывод без подмены его вероятностью вредоносности. |
| dynamic_validation | Exceptions, handler hits, VEH/VCH/UEF, RtlFunctionTable, timeout, exit, dropped events. | Связывает статическую гипотезу с runtime-наблюдением. |

## C.6 Честная защитная формулировка ограничений

Правильная защитная позиция: EH/Unwind Analyzer является специализированным, объяснимым и воспроизводимым инструментом анализа EH/Unwind-метаданных Windows PE32+ AMD64. Он строит карту .pdata/.xdata, выявляет структурные и контекстные аномалии, формирует evidence и помогает реверс-инженеру быстрее перейти к ручной проверке. Он не заменяет весь reverse engineering, не является sandbox, не является универсальным malware classifier и не исполняет неизвестные файлы в static mode.

Таблица C.6 - Ограничения, которые нужно проговорить на защите

| Область | Граница | Правильное объяснение |
|---|---|---|
| Поддерживаемый формат | PE32+ AMD64 | PE32/x86, ARM64, ELF и Mach-O не входят в текущий scope. |
| Static mode | Не исполняет target | Безопасен для первичной проверки, но не видит runtime-only таблицы. |
| Dynamic mode | Только Windows x64 | Запускает target; для untrusted samples обязательна VM/sandbox. |
| Handler data | Не универсально парсится | Language-specific handler data зависит от handler/runtime. |
| UWOP vs .text | Не сравнивается автоматически | Сопоставление unwind operations с дизассемблированным прологом отнесено к развитию. |
| Score/confidence | Не ML-вероятность | Score - triage aid; confidence - инженерная уверенность, а не процент вредоносности. |